

Министерство науки и высшего образования Российской Федерации

**Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский
политехнический университет»**

Факультет «Электротехнический факультет»

Направление подготовки 09.03.04 «Программная инженерия»

Профиль «Разработка программно-информационных систем»

Кафедра «Информационные технологии и автоматизированные системы»

Допускается к защите
Зав. кафедрой _____ /Р.А. Файзрахманов/
« ____ » _____ 2024 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(дипломная работа/дипломный проект)

на тему: Разработка системы оповещения сотрудников компании

Студент: _____ Мехоношин Владислав Антонович
(подпись, дата)

Группа: РИС-20-26

Состав ВКР:

1. Пояснительная записка на ____ стр.
2. Приложения на ____ листах.

Руководитель ВКР:	_____	старший преподаватель Кузнецов Д.Б.
	(подпись, дата)	
Консультант по предметной области:	_____	старший преподаватель Кузнецов Д.Б.
	(подпись, дата)	
Прохождение предзащиты:	_____	старший преподаватель Кузнецов Д.Б.
	(подпись, дата)	
Прохождение нормоконтроля:	_____	ведущий инженер Карлов Д.А.
	(подпись, дата)	
Прохождение проверки на объем заимствований:	_____	инженер Хохрякова Е.А.
	(подпись, дата)	

Пермь 2024

Министерство науки и высшего образования Российской Федерации

**Федеральное государственное автономное образовательное учреждение
высшего образования**

**«Пермский национальный исследовательский
политехнический университет»**

Кафедра информационных технологий и автоматизированных систем

УТВЕРЖДАЮ

Зав. кафедрой

_____/Р.А. Файзрахманов/

« ____ » _____ 2023 г.

З А Д А Н И Е

на выполнение выпускной квалификационной работы

Фамилия, И.О. Мехоношин Владислав Антонович

Факультет Электротехнический Группа РИС-20-2б

Начало выполнения работы 2.10.2023 г.

1. Форма и наименование темы ВКР: (дипломный проект/дипломная работа)

Разработка системы оповещения сотрудников компании

(Утверждена приказом по университету от « ____ » _____ 20__ г. № ____)

2. Исходные данные: Задание на проектирование от компании Веб Депо

3. Содержание пояснительной записки:

- а) цель исследования: повышение эффективности внутрифирменной коммуникации и оперативного информирования персонала о важных событиях и изменениях;
- б) задачи: обследовать существующие типовые процессы сотрудников компаний: об информировании о происшествиях, задачах и встречах, проанализировать существующие системы оповещения, сформулировать требования к системе оповещения сотрудников, разработать техническое задание на создание системы оповещения, разработать прототип системы, функциональное тестирование;
- в) объект исследования: объектом исследования является процесс оповещения сотрудников компании;
- г) предмет исследования: клиент-серверная программная веб-система, обеспечивающая отправку, просмотр и управление оповещениями;
- д) ожидаемые результаты: типовый программный веб-продукт для управления оповещениями сотрудников крупных компаний, позволяющий производить массовую рассылку оповещений по отделам, должностям и отдельным сотрудникам, входящие уведомления сразу возникают на экране в виде всплывающего окна предоставляя возможность для незамедлительной реакции сотрудника, предусмотрено выполнение различных сценариев связанных с реакциями и сбора статистических данных для предоставления отчетов по отдельным оповещениям.

4. Дополнительные указания: Система должна быть разработана с использованием веб технологий, уведомления должны отображаться на рабочем столе вне зависимости от положения браузера.

5. Основная литература:

1. Орлов С.А., Цилькер Б.Я. Технологии разработки программного обеспечения: Учебник для вузов. 4-е изд.

Стандарт третьего поколения. - СПб.: Питер, 2021.

2. Вигерс Карл, Битти Джой Разработка требований к программному обеспечению.

3-е изд., дополненное / Пер. с англ. — М.: Издательство «Русская редакция»;

СПб.: БХВ Петербург, 2014.

Руководитель ВКР:

старший преподаватель кафедры ИТАС

(должность)

(подпись)

/ (фамилия, инициалы) /

Задание получил:

2.10.2023 г.

(дата)

(подпись студента)

/ (фамилия, инициалы) /

КАЛЕНДАРНЫЙ ГРАФИК ВЫПОЛНЕНИЯ ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЫ

Наименование этапа	Объем (в%)	Начало	Конец
1. Изучить существующие типовые процессы сотрудников компаний	15	02.10.23	30.11.23
2. Проанализировать существующие системы оповещения	15	01.12.23	31.12.23
3. Сформулировать требования к системе оповещения сотрудников	15	01.01.24	29.02.24
4. Разработать техническое задание на создание системы оповещения	15	01.03.24	31.03.24
5. Разработать прототип системы и выполнить модульное и функциональное тестирование его работоспособности	20	01.04.24	30.04.24
6. Оформление пояснительной записки	20	01.05.24	20.05.24
7. Представление работы на проверку и отзыв руководителя квалификационной работы	-	21.05.24	26.05.24
8. Предварительная защита работы	-	27.05.24	05.06.24
9. Прохождение нормоконтроля	-	06.06.24	10.06.24
10. Представление работы на кафедру	-	11.06.24	16.06.24
11. Защита на заседании ГЭК	-	24.06.24	30.06.24

Руководитель ВКР: _____ старший преподаватель кафедры
(подпись, дата) ИТАС Кузнецов Д. Б.

Задание получил: _____ Мехоношин Владислав Антонович
(подпись, дата)

РЕФЕРАТ

Выпускная квалификационная работа бакалавра состоит из 5 разделов, 134 стр., 35 рис., 16 источн., 5 табл., 11 лист.

СИСТЕМА ОПОВЕЩЕНИЙ, ПРОГРАММНАЯ СИСТЕМА, ПРОЕКТИРОВАНИЕ, ВЕБ, СЕРВЕР, КЛИЕНТ

Данная работа посвящена разработке системы оповещения сотрудников компании.

В результате выполнения данной работы были выполнены следующие задачи: обследованы существующие типовые процессы сотрудников компаний: об информировании о происшествиях, задачах и встречах. Проанализированы существующие системы оповещения. Сформулированы требования к системе оповещения сотрудников. Разработано техническое задание на создание системы оповещения. Разработан прототип системы. Выполнено функциональное тестирование.

Таким образом, результатом выполненной работы является типовой программный веб-продукт для управления оповещениями сотрудников крупных компаний, позволяющий производить массовую рассылку оповещений по отделам, должностям и отдельным сотрудникам, входящие уведомления сразу возникают на экране в виде всплывающего окна предоставляя возможность для незамедлительной реакции сотрудника, предусмотрено выполнение различных сценариев связанных с реакциями и сбора статистических данных для предоставления отчетов по отдельным оповещениям.

СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	8
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	9
ВВЕДЕНИЕ.....	10
1 Анализ	12
1.1 Анализ предмета автоматизации	12
1.2 Обзор существующих решений.....	13
1.3 Анализ возможностей рассмотренных приложений	20
1.4 Анализ требований.....	21
1.5 Вывод по разделу	23
2 Проектирование.....	24
2.1 Архитектура системы	24
2.2 Функциональные возможности	26
2.3 Отправка и получение новых уведомлений	27
2.4 Реакции.....	28
2.5 Комментарии	28
2.6 Регистрация новых пользователей	29
2.7 Авторизация и аутентификация.....	30
2.8 База данных.....	33
2.9 Прототип интерфейса	34
2.10 Вывод по разделу	36
3 Разработка	37
3.1 База.....	37
3.2 Сервер.....	40
3.2.1 Библиотеки.....	41
3.2.2 Файл настроек.....	44
3.2.3 Взаимодействие с базой	45
3.2.4 Принцип обработки запросов	46
3.2.5 Промежуточные функции	48
3.2.6 Аутентификация и регистрация.....	49

3.2.7 Оповещения	50
3.3 Браузерная часть.....	53
3.3.1 Библиотеки.....	54
3.3.2 Архитектура.....	58
3.3.3 Аутентификация.....	59
3.3.4 Уведомления.....	60
3.3.4.1 Меню	61
3.3.4.2 Список оповещений.....	62
3.3.4.3 Уведомления о новых оповещениях	62
3.3.4.4 Создание оповещений.....	63
3.4.4 Отправленные оповещения.....	64
3.4.5 Регистрация.....	65
3.5 Функциональное тестирование.....	66
3.6 Выводы по разделу.....	70
4 Экономический раздел	71
4.1 Техничко-экономическое обоснование проекта.....	71
4.2 Расчет экономического эффекта.....	71
4.3 Выводы по разделу.....	71
5 Защита информации.....	72
5.1 Определение класса защищённости.....	72
5.2 Обеспечение требований о защите.....	73
5.3 Выводы по разделу.....	74
ЗАКЛЮЧЕНИЕ	75
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	76

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Express.js — это минималистичный и гибкий веб-фреймворк для Node.js, предназначенный для создания серверных приложений и API.

Node.js — это открытая серверная среда выполнения, построенная на движке V8. Она позволяет запускать JavaScript-код вне браузера, что делает возможным создание серверных приложений и сервисов.

PostgreSQL — открытая реляционная система управления базами данных (СУБД), которая поддерживает расширяемость и соответствие стандартам SQL.

Vue.js — JavaScript-фреймворк для создания пользовательских интерфейсов. Он позволяет разработчикам создавать веб-приложения, используя декларативный и компонентный подход.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

БД – база данных

ВКР – выпускная квалификационная работа

ПО – программное обеспечение

СУБД – система управления базами данных

IP – числовой идентификатор компьютера в сети.

ВВЕДЕНИЕ

Компания ООО «Веб Депо» занимается разработкой платформенных решений для предприятий в том числе территориально распределённых. Одной из решаемых задач является оперативное взаимодействие сотрудников.

Таким образом было решено разработать систему уведомления сотрудников компании. Данный продукт направлен на продажу различным компаниям так что он должен быть конкурентоспособен.

На данный момент существуют системы контроля задач, предоставляющие функционал рассылки уведомлений. Но для некоторых компаний функционал таких систем может оказаться избыточным. А если для массовых уведомлений сотрудников использовать мессенджеры или рассылку сообщений по электронной почте, то это исключает возможность контроля осведомлённости сотрудников.

Таким образом разработка системы, специализирующейся на уведомлении сотрудников о важных событиях и происшествиях с возможностью контроля осведомленности считается **актуальной**.

Объектом исследования является процесс оповещения сотрудников компании.

Предметом исследования - клиент-серверная программная веб-система, обеспечивающая отправку, просмотр и управление оповещениями.

Целью является оповещение сотрудников компании с целью повышения эффективности внутрифирменной коммуникации и оперативного информирования персонала о важных событиях и изменениях.

Для достижения цели поставлены следующие **задачи**:

- обследовать существующие типовые процессы сотрудников компаний: об информировании о происшествиях, задачах и встречах;
- проанализировать существующие системы оповещения;
- сформулировать требования к системе оповещения сотрудников;

- разработать техническое задание на создание системы оповещения;
- разработать прототип системы;
- выполнить функциональное тестирование;

1 Анализ

1.1 Анализ предмета автоматизации

Система оповещения сотрудников компании — это инструмент, который используется для передачи сообщений сотрудникам компании о различных важных событиях, таких как: встречи, задачи, происшествия и т.д.

Конкретная цель системы оповещения сотрудников компании заключается в повышении эффективности информирования и контроле осведомленности персонала. В том, чтобы человек не тратил время на поиск конкретного сообщения в чате мессенджера, не тратил время на формулирование ответа, а мог просто нажать на кнопку в появившемся перед ним окне. А руководство могло быть уверенным в осведомленности своих сотрудников.

Основным требованием заказчика было создание конкурентоспособной программной системы для массовой рассылки оповещений, предоставляющей возможность реакций на пришедшие уведомления в условиях ограничений на установку приложений и доступа к внешней сети.

Под «реакцией» подразумевается кнопка, расположенная под уведомлением. По нажатию на данную кнопку будет происходить отправка уведомления, привязанного к ней. Это уведомление может быть направленно как одному человеку, например автору, так и нескольким.

Благодаря реакциям можно значительно увеличить эффективность оповещения сотрудников компании. Например, можно отправить всем сотрудникам одного из отделов вопрос о наличии какой-либо неполадки и на положительный ответ автоматически направить просьбу об исправлении ответственному за это человеку. Таким образом можно значительно ускорить эффективность внутрифирменного взаимодействия.

Также благодаря тому, что системе известно была ли произведена реакция можно однозначно отслеживать осведомленность сотрудников. А на

основании разницы времени отправки и времени прочтения выявлять скорость реакции сотрудников на те или иные события.

1.2 Обзор существующих решений

Для оценки конкурентоспособности создаваемого продукта рассмотрим существующие решения, включающие в себя возможность массовой рассылки уведомлений:

- Projecto от ООО "Проджекто"
- MyChat от NetworkSoftwareSolutions
- Microsoft Teams

Начнем с Projecto. Данный продукт способен вести документооборот, следить за действиями персонала, выполнением поставленных задач и своевременно информировать сотрудников об изменениях и нештатных ситуациях.

Для информирования сотрудников Projecto предоставляет следующие возможности:

- Лента новостей. Во вкладке «Новости» отображается информация, касающаяся изменений в рабочем процессе сотрудника;
- Объявления. Нестандартные, создаваемые вручную оповещения, несущие в себе текстовую информацию с возможностью прикрепления к ним различных документов, изображений и опросов;
- Уведомления. В них отображается наиболее важная для сотрудника информация. Они помещаются выше всех остальных сообщений и могут требовать от сотрудника незамедлительной реакции.

Интерфейс окна уведомлений представлен на рисунке 1.

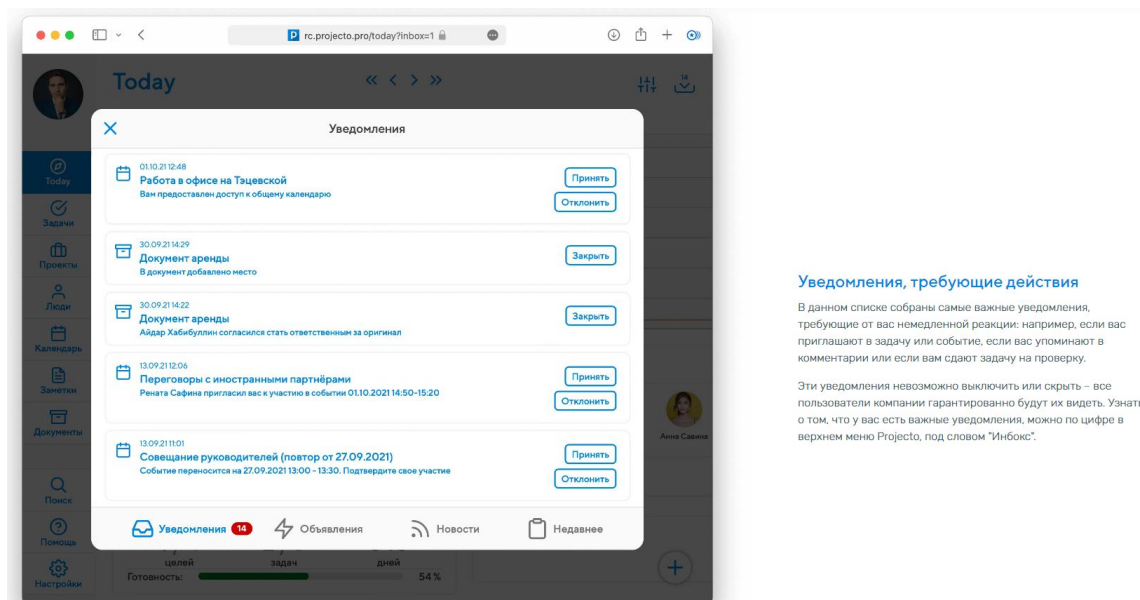


Рисунок 1 – Интерфейс окна уведомлений

Таким образом большая часть уведомлений являются авто генерируемыми. Для нестандартных уведомлений используется система объявлений. В которой не было обнаружено возможности включения в них реакций. Интерфейс создания объявлений показан на рисунке 2.

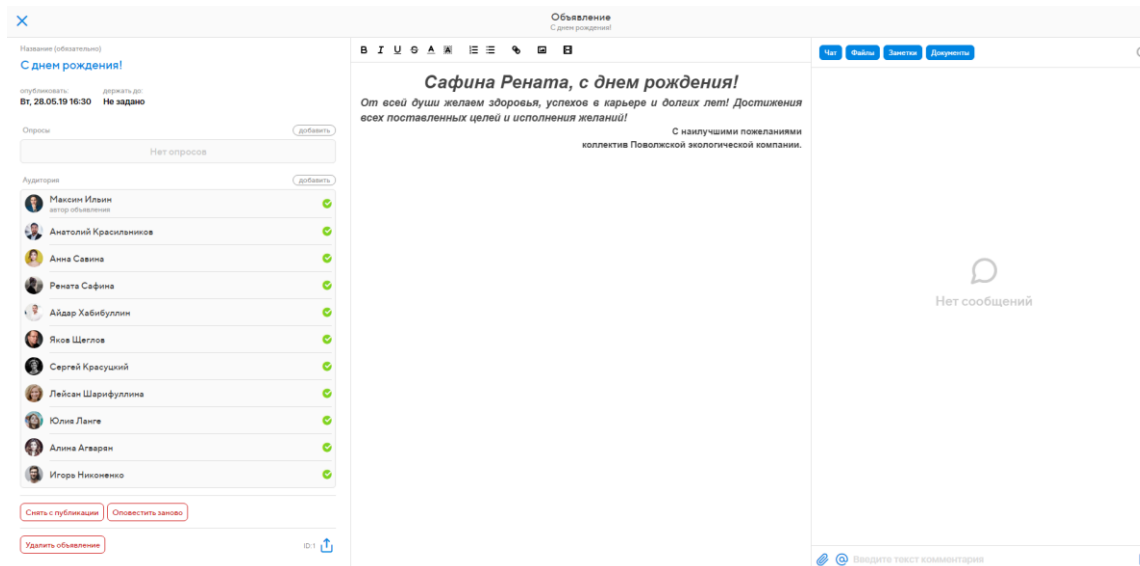


Рисунок 2 – Интерфейс окна создания объявлений

Далее рассмотрим приложение MyChat. Данный продукт является клиент-серверным чатом с функциями массовой рассылки, встроенной Kanban доской, доской объявлений, аудио и видео чатом.

Если Projecto был цельным web приложением, то MyChat является нативным приложением, работающим под различными платформами. Также некоторые из его компонентов, например Kanban доска и форум являются отдельными веб приложениями.

Для массового оповещения сотрудников в MyChat используются массовая рассылка и доска объявлений.

Массовая рассылка представляет собой отправку сообщений нескольким пользователям. При отправке оповещений можно задать получателей, время жизни уведомления и получение уведомлений о прочтении. Интерфейс создания уведомлений показан на рисунке 3.

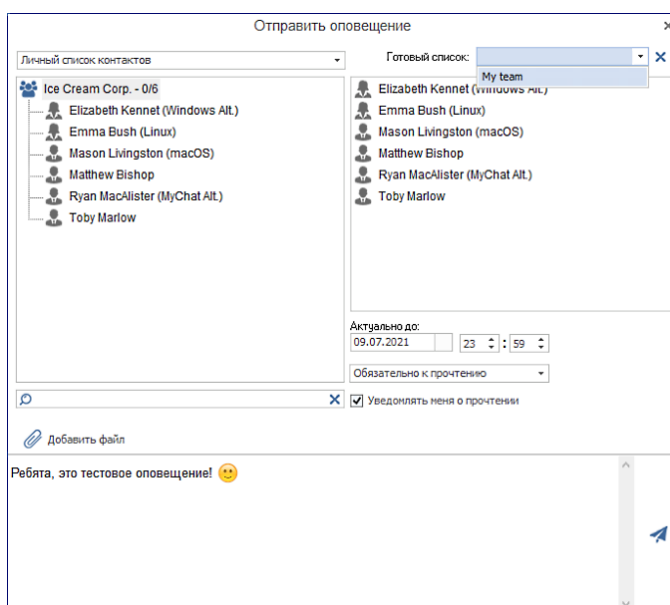


Рисунок 3 – Интерфейс окна создания уведомлений

Отправленное уведомление отображается в нижней части чата с отправителем и не предлагает никаких возможностей для реакции. Пример отправленного уведомления показан на рисунке 4 и выделен желтым цветом.

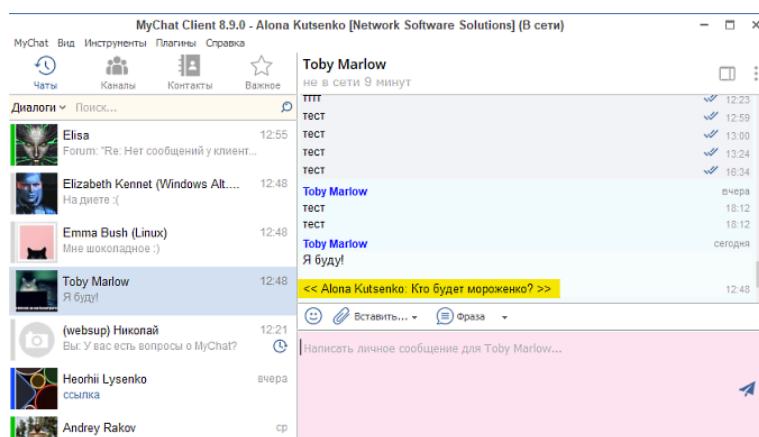


Рисунок 4 – Пример полученного уведомления

MyChat предоставляет возможность контроля осведомленности пользователей во вкладке «История сообщений». Интерфейс данной вкладки показан на рисунке 5.

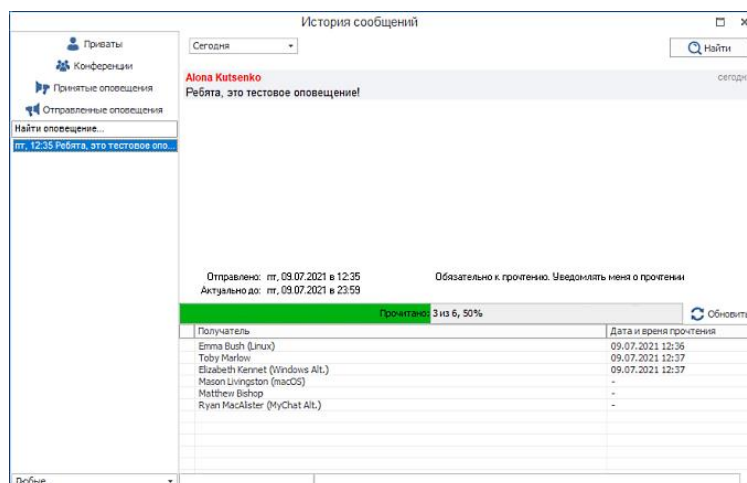


Рисунок 5 – Интерфейс просмотра информации об уведомлении

Доска объявлений является отдельной вкладкой, в которой находится список сообщений, видимый всем пользователям сервера. Отправляемое сообщение можно разместить выше всех остальных, а также задать ему время жизни. Интерфейс отправки таких сообщений показан на рисунке 6.

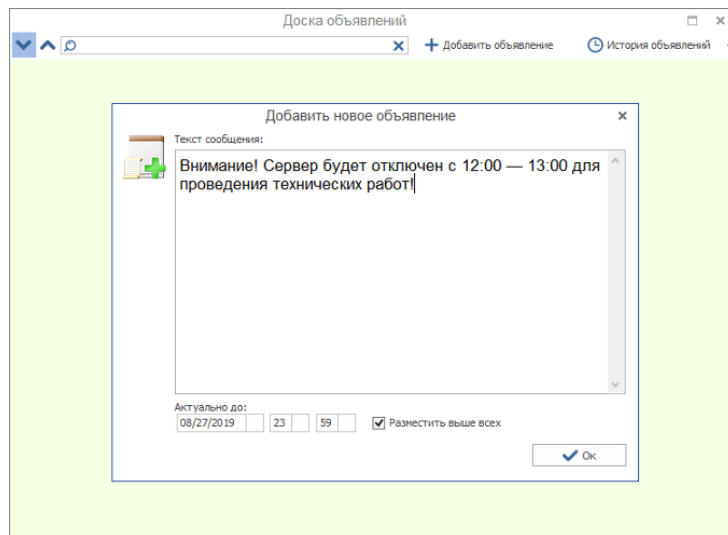


Рисунок 6 – Интерфейс окна создания объявлений

Также в приложении MyChat реализована интеграция на основе предоставляемого разработчиком API.

Рассмотрим Microsoft Teams. Данный продукт представляет собой корпоративный мессенджер. Разработанный компанией Microsoft он включает в себя чат, видеоконференции, хранение файлов, а также интеграцию собственных и сторонних приложений.

Начнем с вкладки «Действия» данная вкладка отображает пропущенные вызовы, приглашения, упоминания пользователя в чатах, новые сообщения и так далее. Вид данной вкладки представлен на рисунке 7.

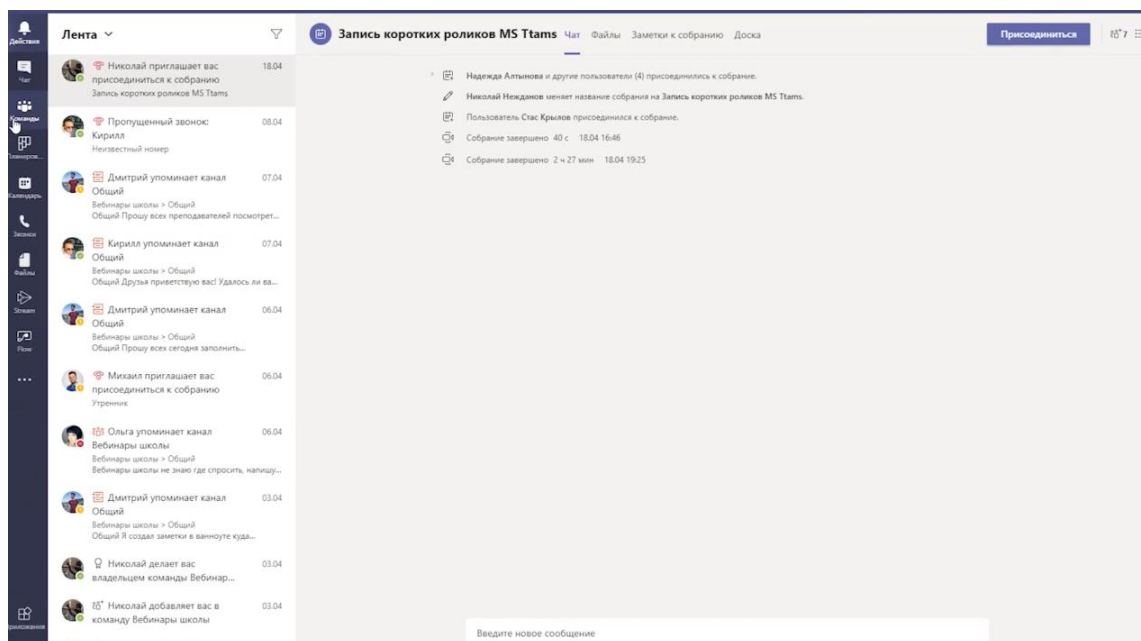


Рисунок 7 – Интерфейс вкладки «Действия»

Вкладка «Чат» в Microsoft Teams представляет собой место, где пользователи могут вести прямые переписки с коллегами или в группах. Это ключевой элемент для общения в рамках организации или проектной группы. Интерфейс данной вкладки показан на рисунке 8.

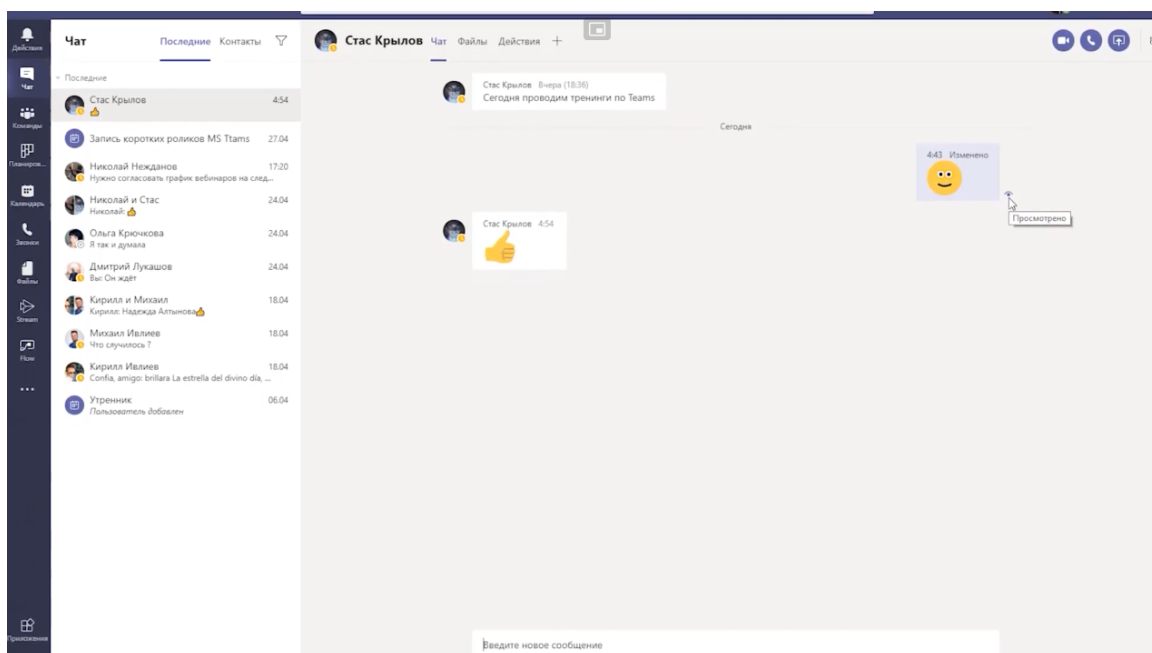


Рисунок 8 – Интерфейс вкладки «Чат»

Вкладка «Команды» в приложении Microsoft Teams является центральным местом для совместной работы и общения в рамках вашей

организации или проектной группы. Эта вкладка позволяет пользователям создавать, присоединяться и управлять командами и каналами, где происходит большая часть общения и совместной работы. Интерфейс данной вкладки показан на рисунке 9.

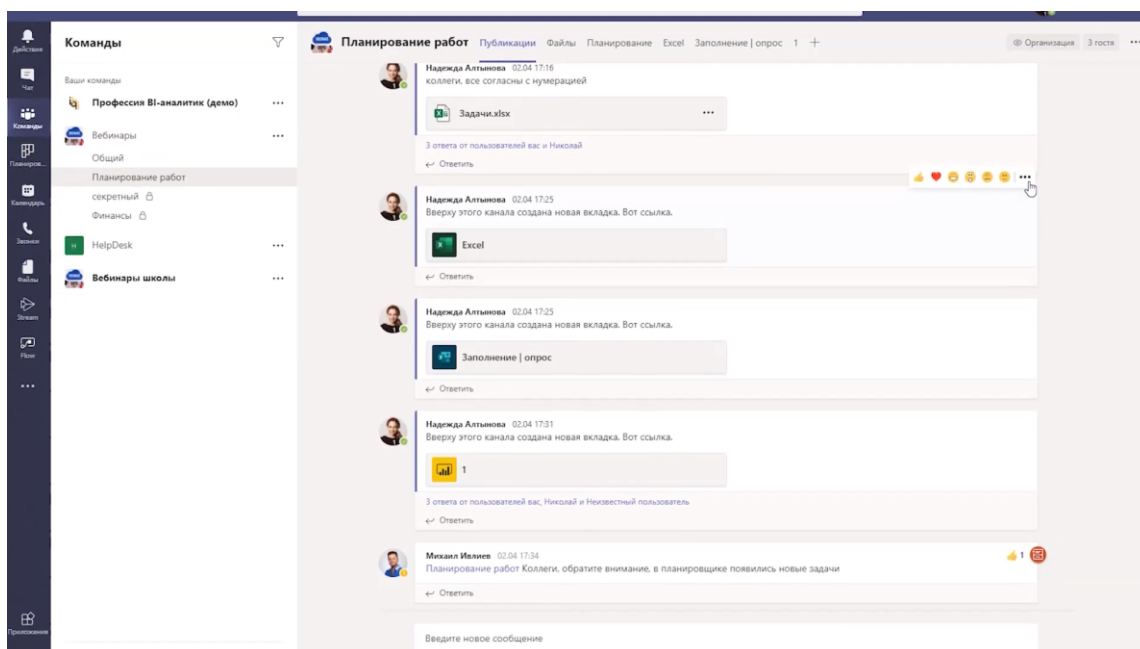


Рисунок 9 – Интерфейс вкладки «Команды»

Microsoft Teams предоставляет широкие возможности для интеграции сторонних приложений, что позволяет пользователям расширять функциональность и адаптировать рабочее пространство под свои потребности. Список возможностей для интеграции:

- Коннекторы и вебхуки. Коннекторы позволяют Teams получать обновления от популярных веб-сервисов. Например, можно настроить получение уведомлений о новых постах в блоге, обновлениях в проекте или изменениях в задачах. Вебхуки предоставляют возможность для более глубокой интеграции, позволяя сторонним сервисам отправлять кастомизированные уведомления прямо в каналы или чаты Teams;

- Магазин приложений. Microsoft Teams имеет собственный магазин приложений, где можно найти и установить нужные приложения для

интеграции. Это делает процесс поиска и добавления новых функций очень удобным и быстрым;

– Разработка собственных решений. Для организаций, имеющих специфические потребности, Teams предоставляет платформу для разработки собственных приложений и ботов, которые можно интегрировать непосредственно в рабочее пространство. Microsoft предоставляет мощные инструменты и API для разработчиков.

1.3 Анализ возможностей рассмотренных приложений

Для постановки требований стоит проанализировать рассмотренные продукты относительно возможностей уведомления сотрудников. В таблице 1 выписаны названия продуктов и их возможности

Таблица 1 – Сравнение рассмотренных продуктов

	Projecto	MyChat	Microsoft Teams
Массовая рассылка оповещений	+	+	+/- (только в групповых чатах)
Возможность привязки реакций к уведомлениям	+/- (реакции есть только в заготовленных уведомлениях)	-	-
Возможность контроля осведомленности сотрудников	+	+	+/- (значок о прочтении сообщения)
Возможность интеграции с уже имеющимися системами	+	+	+
Гибкость благодаря настройкам	+	+	+

Продолжение таблицы 1

Взаимодействие сотрудников по средствам сообщений	+	+	+
Возможность использования в условиях запрета на установку дополнительного ПО и доступа во внешние сети	-	-	-

Таким образом Projecto это комплексный продукт, предоставляющий множество возможностей по управлению различными задачами и событиями. Но при этом для использования всех возможностей оповещения сотрудников необходимо использование всего остального функционала приложения. А функционал создания нестандартных не предоставляет возможности контроля осведомленности или реакций.

MyChat предоставляет возможность массовой рассылки оповещений и контроля осведомленности сотрудников. Но у сотрудников нет никакой возможности быстро отреагировать на оповещения кроме написания сообщения автору уведомления.

Microsoft Teams является корпоративным мессенджером не предоставляющим возможностей отправки оповещений помимо созданий массовых чатов или каналов.

В итоге рассмотренные продукты не могут предоставить функционал для создания и массовой рассылки нестандартных уведомлений с возможностью встраивания в них реакций и контролем осведомленности сотрудников в условиях запрета на установку дополнительного ПО и доступа во внешние сети.

1.4 Анализ требований

При разработке требований к программному обеспечению важно учитывать множество факторов, таких как интересы пользователей,

технические ограничения и бизнес-цели. Подчеркивается, что тщательное определение и документирование требований является ключевым этапом в процессе создания качественного программного продукта [1].

Система оповещения сотрудников компании должна обеспечивать повышение эффективности внутрифирменной коммуникации благодаря некоторым ключевым особенностям описанных в таблице 2.

Таблица 2 – Требования к системе

Требование	Описание
Массовая рассылка оповещений	Возможность массовой рассылки уведомлений благодаря делению по отделам и должностям
Возможность привязки реакций к уведомлениям	При создании уведомления должна быть возможность добавить к нему реакцию. То есть кнопку, имеющую отображающийся на ней текст и привязанную к другому уведомлению, которое будет отправлено по нажатию на неё. А при отсутствии привязанных реакций у получателя отображается одна реакция «прочитать».
Возможность контроля осведомленности сотрудников	Контроль осведомленности сотрудников должен производиться по прочтению уведомлений. Уведомление может считаться прочитанным если сотрудник отреагировал на него. Сотрудник, отправивший уведомление должен иметь возможность просмотреть процент и список отреагировавших. Аналитик должен иметь возможность просмотреть ту же информацию по всем уведомлениям в системе.
Возможность интеграции с уже имеющимися системами	Вполне вероятно, что у копания собирающейся интегрировать данный продукт уже есть система учета сотрудников. Поэтому итоговая система должна иметь модульную структуру, позволяющую легко вносить в нее изменения.
Гибкость благодаря настройкам	Из-за того, что создаваемый продукт создается не для конкретной организации. Должна быть возможность занесения в систему списка должностей, отделов, видов уведомлений.

Взаимодействие сотрудников по средствам сообщений	Взаимодействие сотрудников по средствам текстовых сообщений.
Возможность использования в условиях запрета на установку дополнительного ПО и доступа во внешние сети	Возможность установки и использования системы локально без установки дополнительного ПО.

Выполнение описанных выше требований позволит создать конкурентоспособный продукт, выполняющий желаемые заказчиком функции. Также на основе описанных требований было составлено техническое задание, представленное в приложении А.

1.5 Вывод по разделу

В данном разделе было дано описание системы оповещения сотрудников компании. Также были рассмотрены и проанализированы возможности существующих систем, включающих в себя функционал создаваемой системы.

На основе предоставляемого существующими системами функционала, а также желаний заказчика был составлен список требований, выполнение которых позволит создать конкурентное решение, увеличивающее эффективность внутрифирменной коммуникации.

2 Проектирование

2.1 Архитектура системы

Система — множество элементов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство.

В процессе разработки программного обеспечения важно учитывать современные методологии и подходы. Одним из требований к системе является возможность работы в условиях запрета установки дополнительного ПО и доступа во внешние сети. Наиболее простой способ обеспечения доступности приложения без установки какого-либо ПО — это создание веб-приложения. Веб-приложения работают в браузерах, которые доступны на большинстве устройств, включая настольные компьютеры, ноутбуки, планшеты и смартфоны. Это означает, что приложение будет работать из браузера любого устройства, имеющего доступ к серверу на котором установлена система [2].

Таким образом было принято решение о создании клиент серверного веб приложения, состоящего из трёх основных компонентов. Это база данных, сервер и клиентская часть, являющаяся сайтом. Схема будущей системы представлена на рисунке 10.

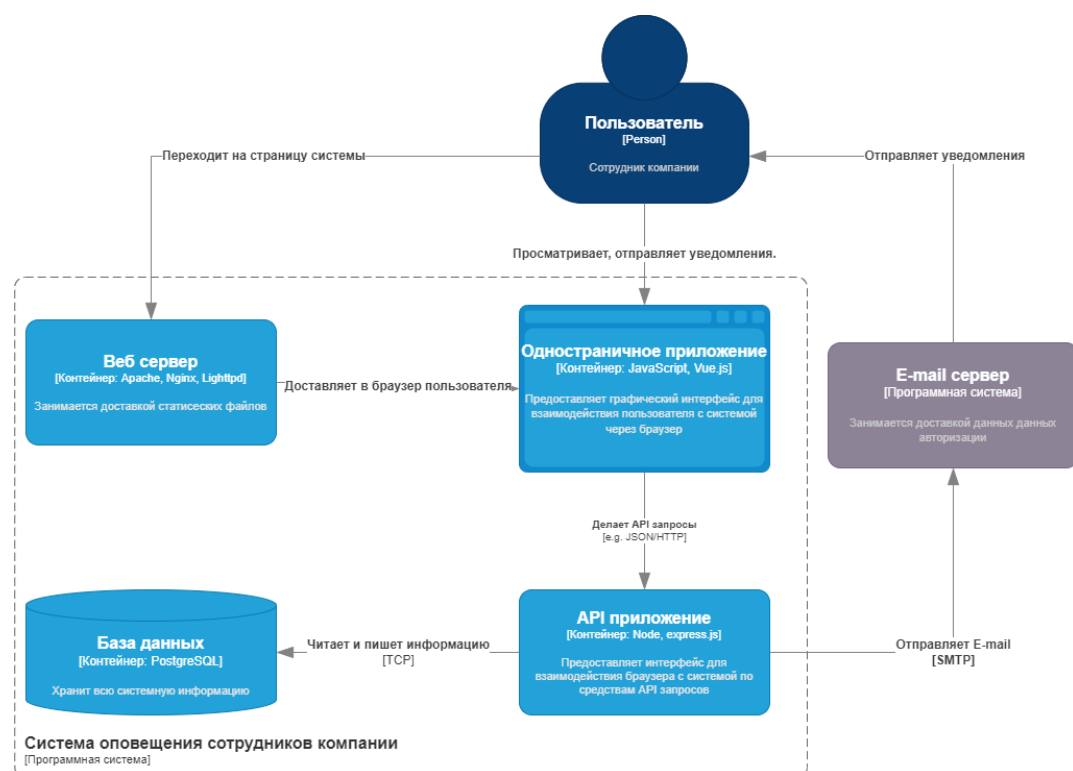


Рисунок 10 – Диаграмма архитектуры системы

База данных является хранилищем всех данных системы. В качестве СУБД используется PostgreSQL.

Сервер является посредником между клиентом и базой данных. В его основе технология Node.js с применением фреймворка express.js. Основной его функцией является контроль доступа пользователей к данным и предоставление удобного api. Сервер состоит из следующих частей:

1. Routers – объекты, контролирующие данные, которые будут возвращены при обращении по определенному пути;
2. Controllers – объекты, занимающиеся формированием и проверкой данных;
3. Services – объекты, описывающие функции обеспечивающие делительную функциональность;
4. Middlewares - объекты, описывающие функции выполняющиеся сразу после обращения к серверу;
5. Exceptions – объекты, описывающие ошибки.

Браузер, то есть клиентская часть системы построена на основе технологии Node.js с применением фреймворка Vue.js. Данный фреймворк позволяет строить приложение из отдельных компонент соединяя и переиспользуя их на различных страницах [5].

Клиентская часть состоит из следующих частей:

1. Components – компоненты, являющиеся отдельными частями интерфейса;
2. Pages – страницы, объединяют в себе компоненты;
3. Router – роутеры, определяют какую страницу отобразить;
4. Api – обработка запросов к серверу;
5. Store – глобальное хранилище данных.

2.2 Функциональные возможности

На основе выдвинутых в пункте 1.4 требований была разработана use-case диаграмма, описывающая взаимодействие между пользователями (актерами) и системой. Диаграмма представлена на рисунке 11.

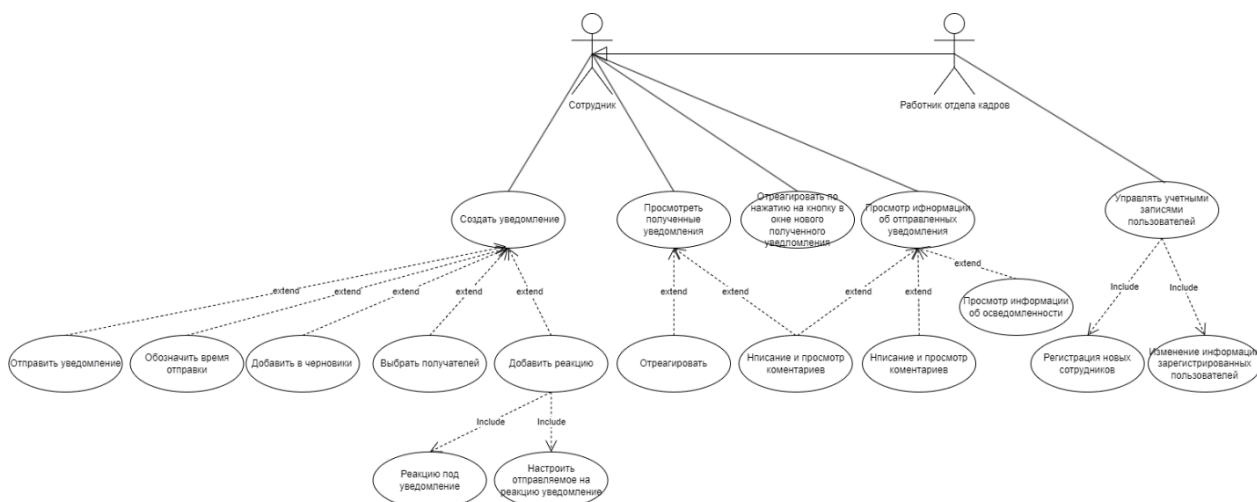


Рисунок 11 – Use-case диаграмма

В системе должно быть выделено 2 основные роли: сотрудник и работник отдела кадров. Каждый сотрудник должен иметь возможность создать, сохранить в черновик и отправить уведомление, просмотреть список отправленных уведомлений, а также просмотреть и отреагировать на

полученные им уведомления. А работник отдела кадров регистрировать в системе новых сотрудников и изменять их информацию.

2.3 Отправка и получение новых уведомлений

Создание новых уведомлений происходит в отдельной форме, предоставляющей поля для ввода всех необходимых данных. При попытке отправления будет происходить валидация данных и формирование объекта для отправки на сервер. Получив запрос на отправку уведомления, сервером произведет аутентификацию и валидацию. Далее сервер отправит запрос к БД на добавление новых записей. После получения сервером ответа об успешном добавлении данных на клиент будет отправлен ответ с кодом статуса 200. После получения ответа форма будет закрыта. Для демонстрации процессов отправки и получения уведомления была разработана sequence диаграмма, показанная на рисунке 12.

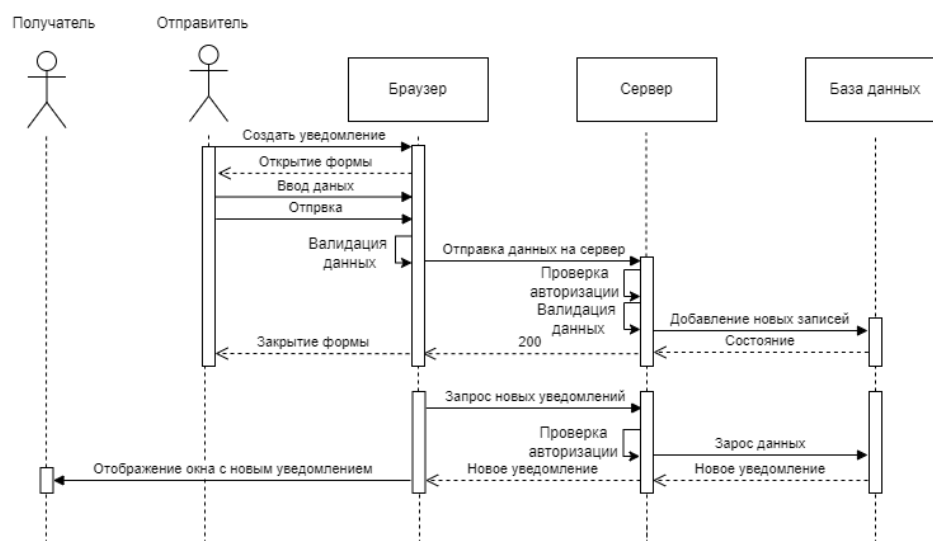


Рисунок 12 – Получение и отправка уведомлений

Обновление данных на клиенте происходит путем периодического опроса. А отслеживание новых записей происходит благодаря хранению id последней полученной записи, то есть в запросе к базе данных будет отслеживаться id получателя id записи об отправлении уведомления.

2.4 Реакции

Как уже упоминалось ранее под «реакцией» подразумевается кнопка, расположенная под уведомлением. По нажатию на данную кнопку будет происходить отправка уведомления, привязанного к ней. Это уведомление может быть направленно как одному человеку, например автору, так и нескольким. Происходящие при реакции процессы показаны на рисунке 13.

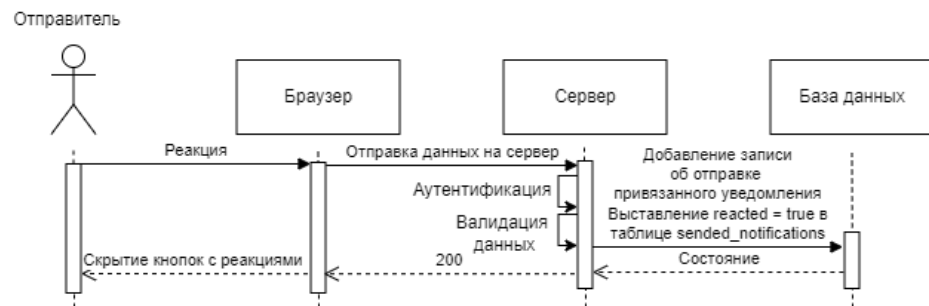


Рисунок 13 – Реакции

Каждый получивший уведомление пользователь имеет возможность нажать на одну из представленных под ним кнопок и только один раз. После чего все представленные под уведомлением кнопки будут скрыты.

Если к отправленному уведомлению не было привязано ни одной реакции, то под ним появится кнопка «Прочитано». Данная кнопка требуется для контроля осведомленности, в случае если для конкретного уведомления не требуется каких-то особых реакций.

2.5 Комментарии

На случай необходимости дополнительного обсуждения проблемы под каждым из уведомлений находятся комментарии. Они привязаны к конкретному уведомлению, их видит отправитель и каждый получатель уведомления. Сценарий использования комментариев и связанные с ним процессы показан на рисунке 14.

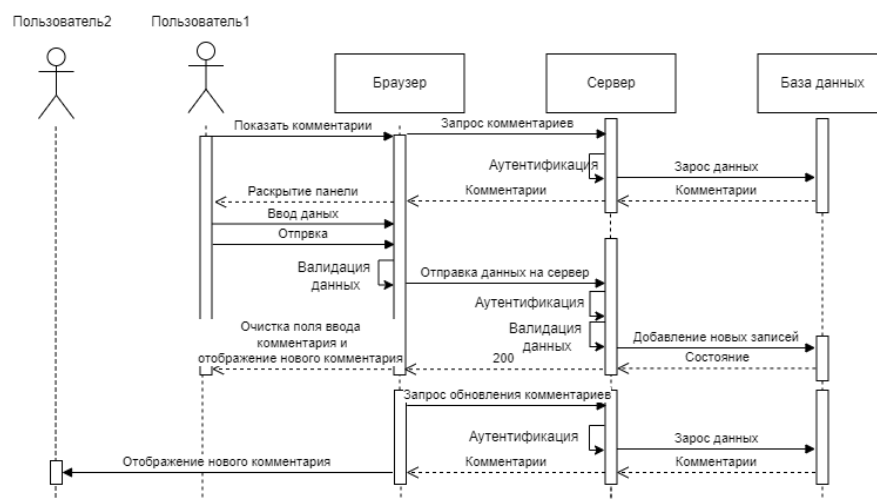


Рисунок 14 – Отправка и получение комментариев

Количество комментариев под уведомлением обновляется автоматически. А при раскрытии комментариев под уведомлением начинают обновляться еще и сами комментарии.

2.6 Регистрация новых пользователей

Регистрация новых пользователей будет происходить сотрудником отдела кадров. То есть на стороне клиента и сервера будет происходить проверка соответствующих ролей. Данные авторизации будут состоять из логина и пароля. Логин по умолчанию будет предоставленный сотрудником адрес электронной почты, а пароль будет сгенерирован на стороне сервера, после чего на предоставленную электронную почту будет отправлено сообщение со ссылкой на систему и данными для авторизации в ней. Процесс регистрации новых сотрудников показан на рисунке 15.

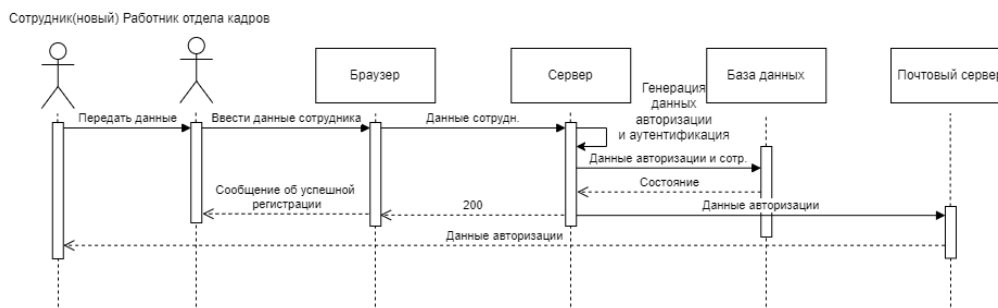


Рисунок 15 – Регистрация новых пользователей

Учитывая, что регистрация пользователей происходит с применением ручного ввода данных, должна быть предусмотрена возможность корректировки возможных ошибок. Для этого была предусмотрена возможность изменения данных пользователей. Интерфейс для этого расположен на той же странице сразу после формы для регистрации данных новых сотрудников. Процессы, происходящие при изменении данных показаны на рисунке 16.



Рисунок 16 – Изменение данных пользователей

Одновременно можно изменить данные нескольких пользователей, но отправка изменений на сервер будет происходить только по нажатию на соответствующую кнопку.

2.7 Авторизация и аутентификация

Авторизация будет построена на базе JSON web token. По сути JWT – json объект, представляющий из себя кодированную строку, включающую в себя 3 части:

1. Header(Заголовок) – содержит в себе данные о том, как должна вычисляться JWT подпись;

2. Payload(Полезная нагрузка) – может содержать в себе любые данные. Например id пользователя, его роли и так далее;
3. Signature(Подпись) – хэш созданный алгоритмом HMACSHA256 состоящий из кодированных алгоритмом base64url частей header и payload сложенных через точку и переданного HMACSHA256 секретного ключа.

Далее все 3 части складываются следующим образом: `encodeBase64Url(header) + '.' + encodeBase64Url(payload) + '.' + encodeBase64Url(signature)`.

Таким образом декодировав 1 часть можно узнать алгоритм вычисления подписи, декодировав 2 можно узнать информацию о том кто отправил запрос, а если вычислить подпись еще раз с помощью первых 2 частей и сравнить с 3 частью пришедшего токена, то можно подтвердить его пригодность. Важно отметить, что подделать подпись или изменить содержимое самого токена без секретного ключа не выйдет. Также в части payload можно хранить время жизни ключа.

Для демонстрации процесса авторизации была разработана activity диаграмма, показанная на рисунке 17.

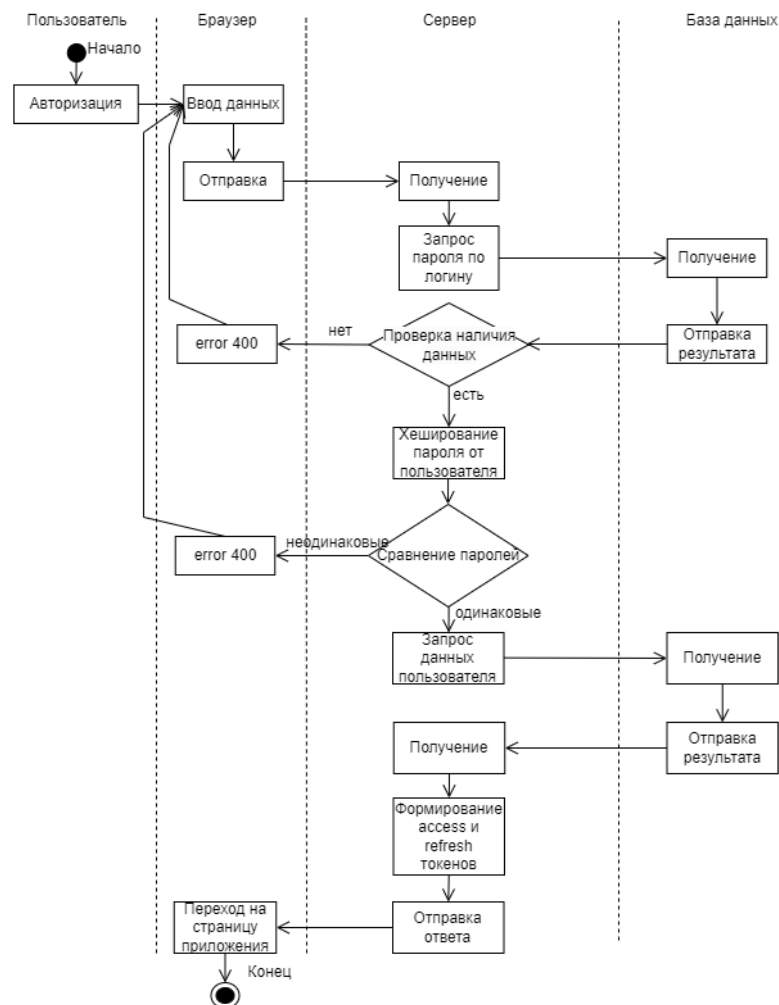


Рисунок 17 – Процесс авторизации

Аутентификация происходит путем хеширования отправленного пользователем пароля и его сравнения с сохраненным в базе данных хэшем. При их совпадении происходит формирование пары access и refresh токенов. Refresh токен сохраняется в базе после чего оба токена отправляются на клиент.

Refresh токен нужен для обновления пары токенов и валиден долгое время. Access токен участвует в процессе аутентификации и обычно валиден несколько минут и при истечении его срока действия происходит обновление токенов.

При обновлении токенов срок действия access токена уже истек, старый refresh токен удаляется из базы и после генерации новой пары токенов

происходит сохранение нового refresh токена и отправка сгенерированной пары токенов на клиент.

При переходе на страницу приложения будет произведен запрос на получение необходимых для работы приложения данных. Если в ответ на запрос клиент получает ошибку 401, то клиентом будет произведена попытка обновления токенов и повторение отправленного ранее запроса. Если при обновлении токенов также будет получена ошибка, то пользователь будет перемещен на страницу авторизации.

2.8 База данных

После описания работы основных функций будущей системы была разработана физическая модель базы данных. Сама модель представлена на рисунке 18.

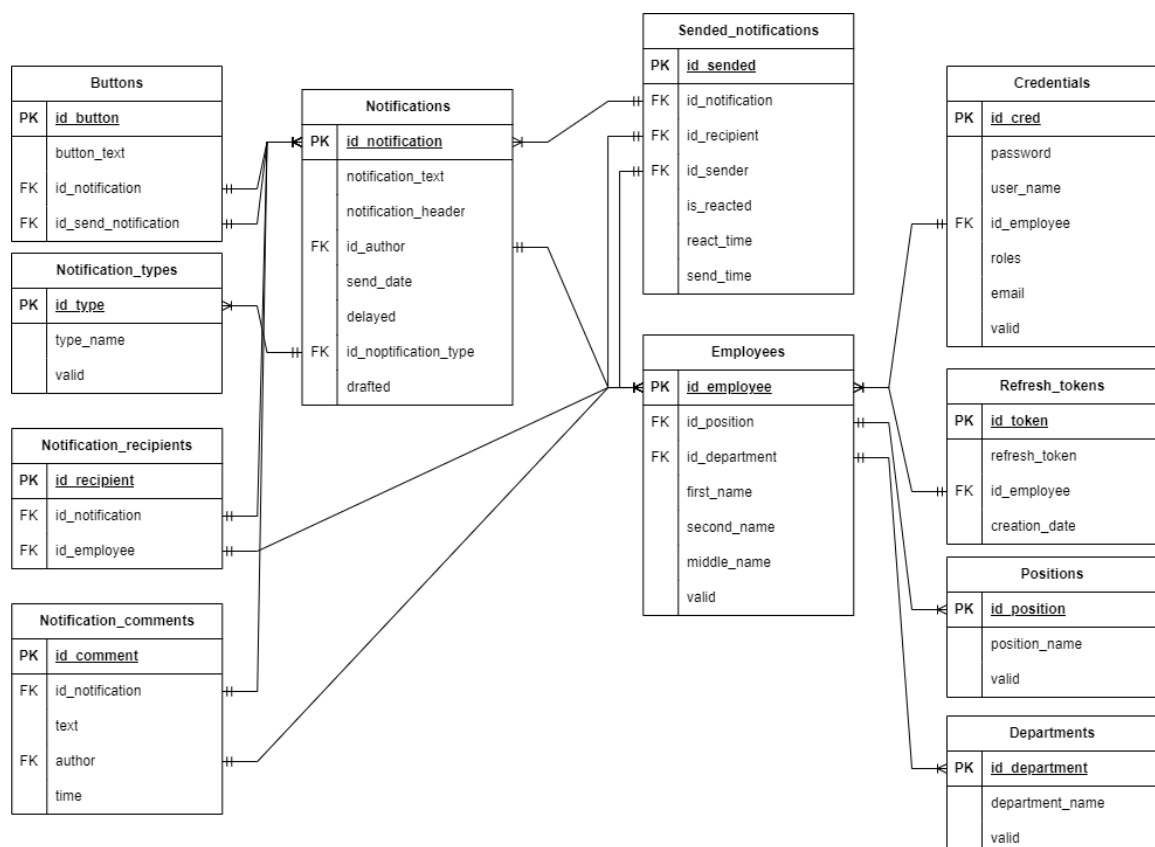


Рисунок 18 – ER диаграмма базы данных

Центральными объектами в базе являются таблицы employees и notifications хранящими данные о пользователях и уведомлениях

соответственно. Дополнительные данные об уведомлениях хранят таблицы recipients и buttons. Стоит отметить, что таблица notification хранит данные обо всех уведомлениях, не важно являются они черновиками или отправлены. При отправлении уведомления в таблице sented_notification хранятся id уведомления, отправителя и получателя. Positions хранит список возможных должностей. Notification_types все возможные типы уведомлений.

2.9 Прототип интерфейса

Также был разработан прототип интерфейса. Разработка происходила с использованием принципов проектирования интерфейса. Прототип главной страницы приложения с отмеченными на нем принципами проектирования показан на рисунке 19.

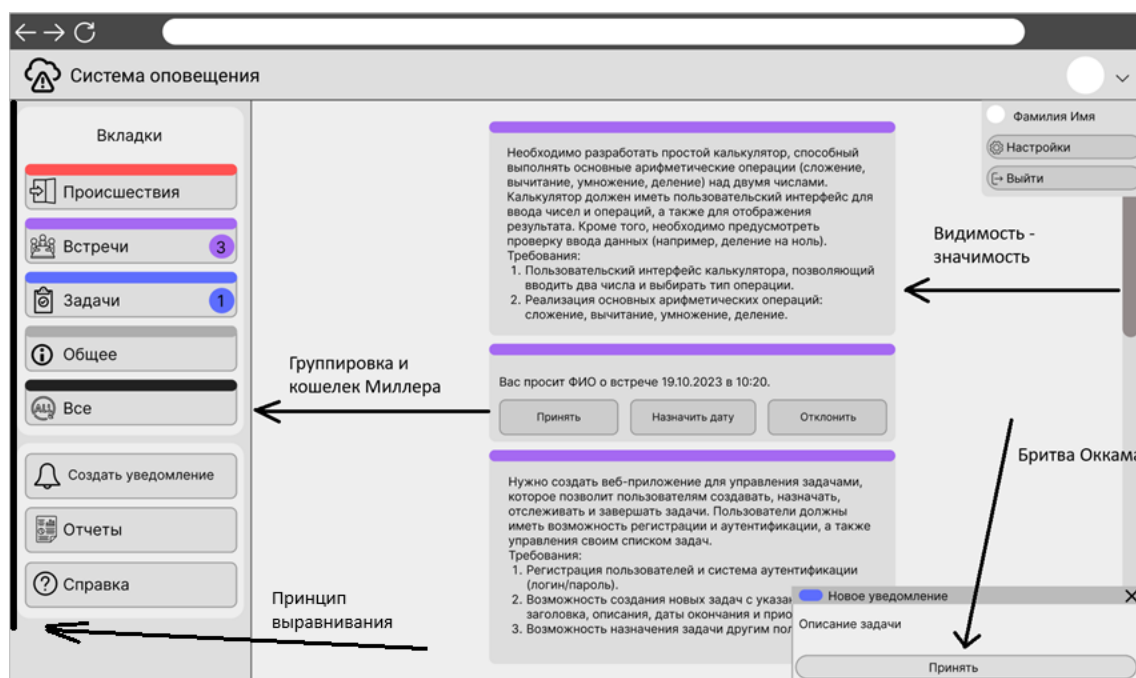
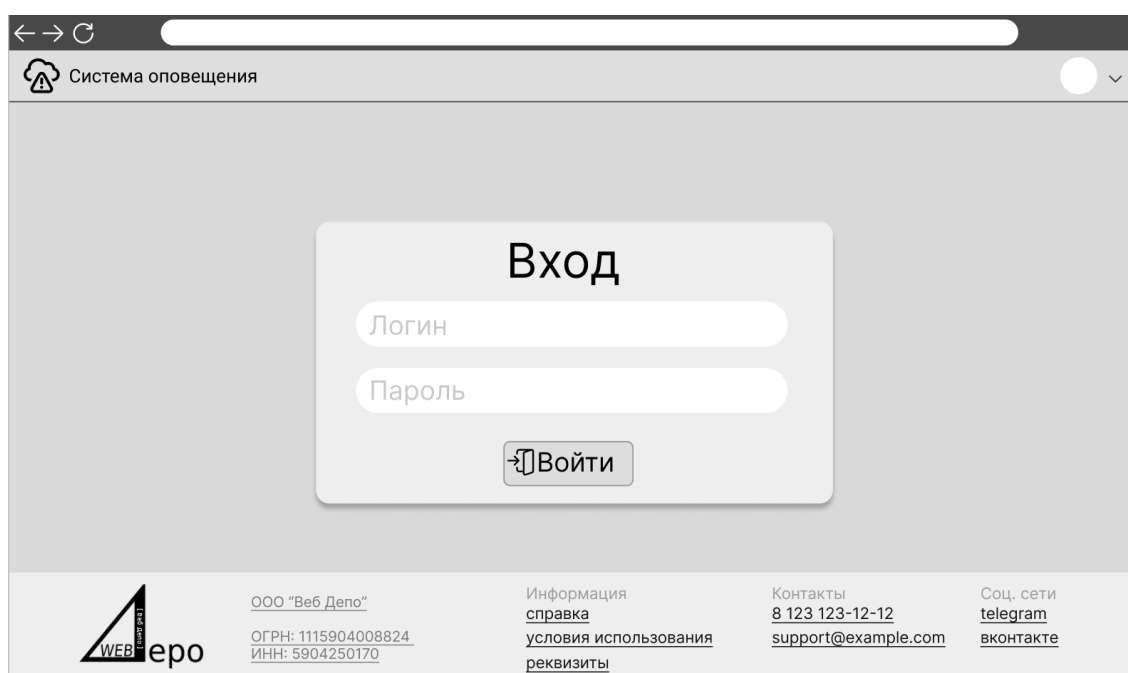


Рисунок 19 – Пример использования принципов проектирования при создании прототипа интерфейса главной страницы

Принцип группировки говорит о том, что экран программы должен быть разделён на ясно выделенные блоки элементов. Принцип кошелька Миллера говорит о том, что элементов в группах должно быть не более семи. Принцип бритвы Оккама о том, что задача должна решаться минимальным количеством действий. Принцип выравнивания от том, что элементы на экране должны

быть выронены. Видимость-значимость о том, что наиболее значимые элементы должны быть внесены на передний план.

Также были разработаны прототипы интерфейса для страницы аутентификации и для окна создания уведомлений. Данные прототипы продемонстрированы на рисунках 20 и 21.



Скриншот прототипа веб-интерфейса для системы оповещения. В центре экрана находится форма входа с заголовком "Вход". Форма содержит два поля ввода: "Логин" и "Пароль", а также кнопку "Войти" с иконкой ключа. В верхней части экрана видна панель браузера с адресной строкой и значком сайта. В нижней части экрана расположен футер с логотипом "Веб-Депо" и контактной информацией.

Система оповещения

Вход

Логин

Пароль

Войти

Веб-Депо

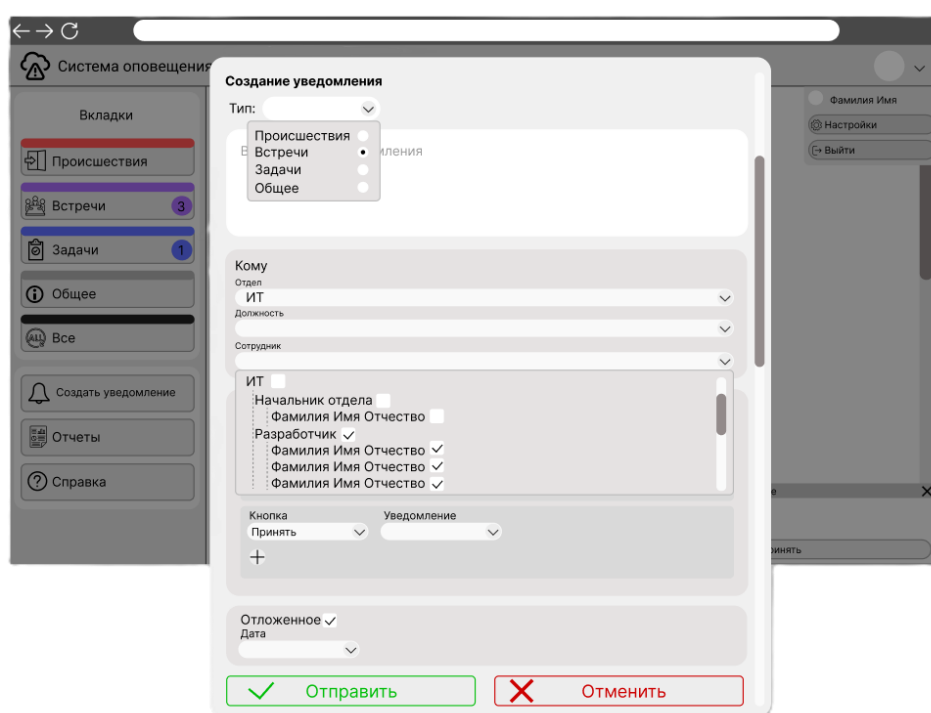
ООО "Веб-Депо"
ОГРН: 1115904008824
ИНН: 5904250170

Информация
справка
условия использования
реквизиты

Контакты
8 123 123-12-12
support@example.com

Соц. сети
telegram
вконтакте

Рисунок 20 – Прототипа интерфейса страницы аутентификации



Скриншот прототипа веб-интерфейса для создания уведомления. В центре экрана находится форма "Создание уведомления". Форма содержит поля для выбора типа уведомления, назначения (кому), даты и времени, а также кнопки "Отправить" и "Отменить". В левой части экрана видна панель с вкладками и меню.

Система оповещения

Вкладки

- Происшествия
- Встречи
- Задачи
- Общее
- Все

Создать уведомление

Отчеты

Справка

Создание уведомления

Тип: ☐ Происшествия ☐ Встречи ☐ Задачи ☐ Общее

Кому:

Отдел:

Должность:

Сотрудник:

ИТ: ☐

Начальник отдела: ☐

Фамилия Имя Отчество:

Разработчик: ☒

Фамилия Имя Отчество: ☒

Фамилия Имя Отчество: ☒

Фамилия Имя Отчество: ☒

Кнопка:

Уведомление:

Отложенное: ☒

Дата:

Отправить

Отменить

Рисунок 21 – Прототип интерфейса создания оповещений

2.10 Вывод по разделу

В данном разделе на основе требований была выбрана архитектура разрабатываемой системы.

Также были продемонстрированы сценарии взаимодействия пользователей с помощью диаграмм последовательности. Далее были описаны основные функции системы и процессы, происходящие при их выполнении с помощью activity диаграмм.

Основываясь на описании функций и процессов разрабатываемой системы, была разработана ER диаграмма базы данных.

С использованием принципов проектирования интерфейсов были созданы прототипы интерфейса системы оповещения. Таким образом была проведена подготовка к реализации системы, позволяющая ускорить процесс разработки и уменьшить количество ошибок.

3 Разработка

3.1 База

Существует 2 способа создания базы данных:

1. Написание sql запросов
2. Использование различных программных средств, предоставляющих графический интерфейс

Основным достоинством первого способа является гибкость, но при этом увеличивается возможное количество ошибок, возникающих при ручном написании кода.

При использовании второго способа основным ограничением является используемое программное обеспечение. Но при этом благодаря интуитивно понятному интерфейсу и визуализации проделанной работы мы можем получить меньшее количество ошибок и увеличение скорости работы.

Учитывая относительную простоту создаваемой базы данных, был выбран второй вариант.

Для используемой в данном проекте СУБД PostgreSQL в пакет установки уже включен инструмент для администрирования баз данных pgAdmin [12]. Данный инструмент предоставляет удобный графический интерфейс и множество возможностей по его использованию, включающих в себя:

- Управление базами данных;
- Мониторинг и диагностику;
- Импорт и экспорт данных;
- Управление правами доступа.

Меню создания таблицы на 6 разделов. Нас интересуют только 3 из них: `general` – ввод названия таблицы, `columns` – описание данных, хранящихся в

таблице, constraints – описание ограничений. Интерфейс раздела columns представлен на рисунке 22.

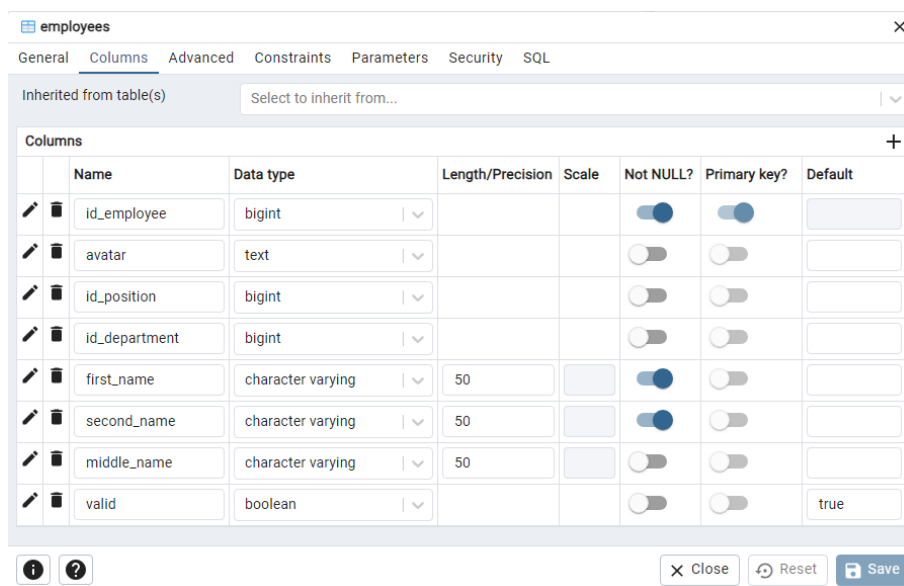


Рисунок 22 – Раздел columns

Также для первичных ключей был добавлен атрибут IDENTITY позволяющий сделать их авто инкрементируемыми. Пример показан на рисунке 23.

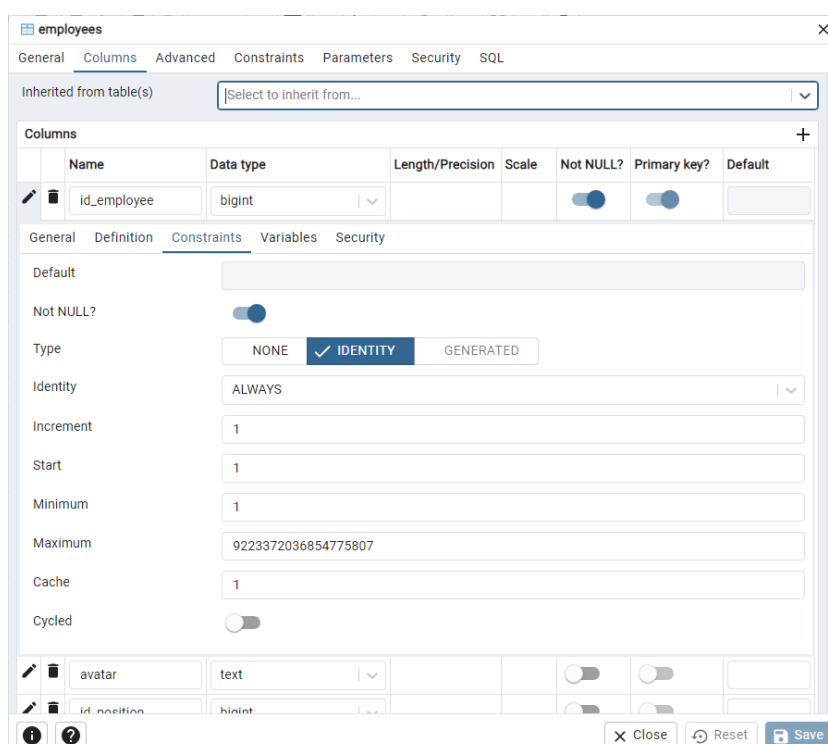


Рисунок 23 – Настройка ограничений

Для внешних ключей в разделе «Constraints» также были прописаны нужные ограничения и действия при выполнении каскадных операций. Пример настройки показан на рисунке 24.

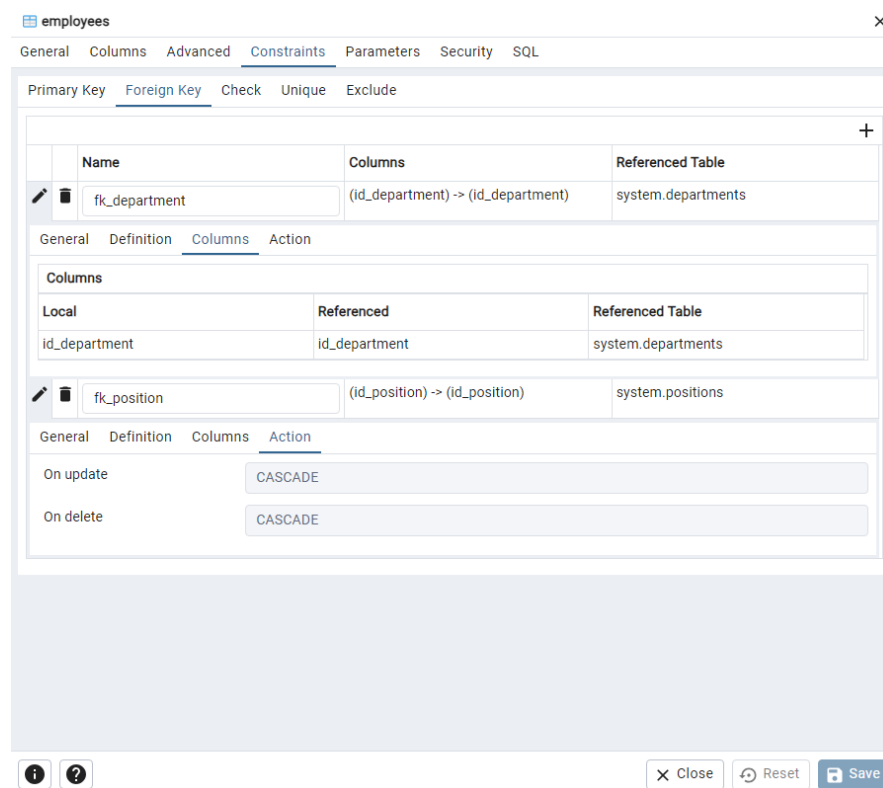


Рисунок 24 – Настройка ограничений для внешних ключей

Таким образом была реализована база данных о проектировании которой написано в параграфе 2.8. После создания всех таблиц pgAdmin предоставляет возможность визуализировать введенные данные в виде ER диаграммы. Пример такой визуализации показан на рисунке 25.

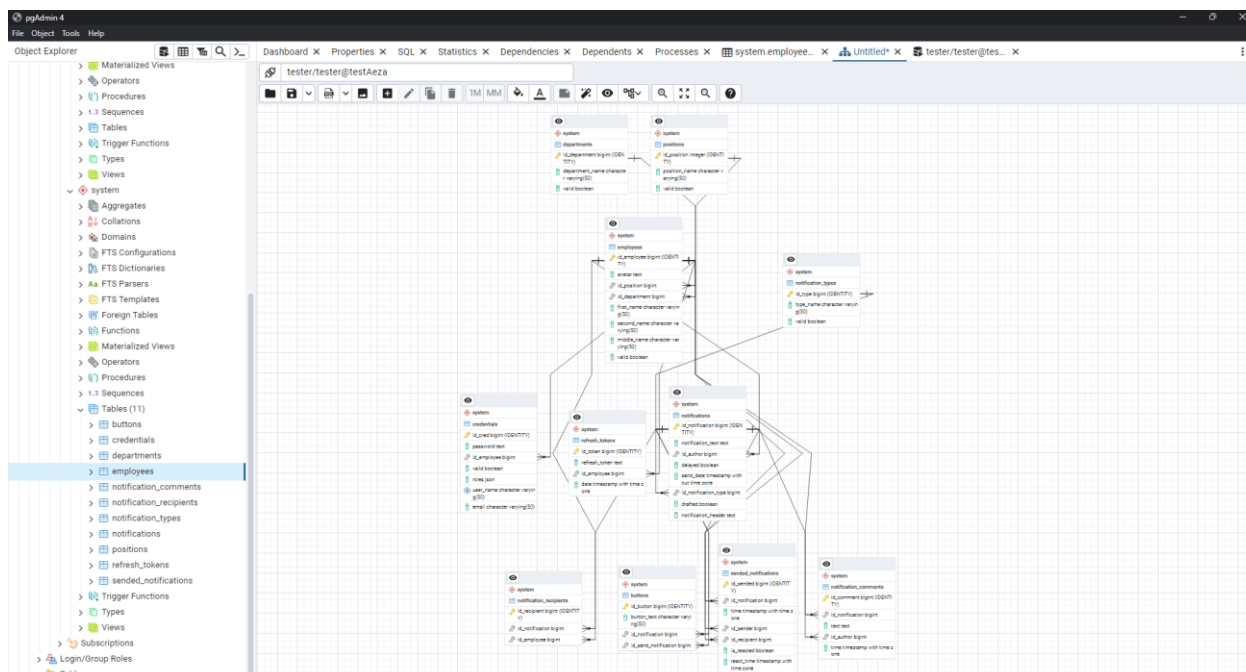


Рисунок 25 – Визуализация с помощью ER диаграммы

И сгенерировать SQL скрипт, создающий таблицы, представленные на диаграмме. Сгенерированный скрипт для базы данных использующейся данной системой представлен в приложении Б.

3.2 Сервер

Для создания сервера использовался фреймворк express.js. Он представляет собой набор полезных инструментов для создания веб-приложений и API [4].

Основными особенностями данного фреймворка являются:

- Предоставление набора функций для создания веб-приложений без излишней сложности;
- Основной компонент обработки запроса – middleware. По своей сути он является функцией, обрабатывающей запросы. Данные функции могут выстраиваться в цепочки выполняя валидацию, изменяя данные в запросе, проводя аутентификацию;
- Предоставление простой, но гибкой системы маршрутизации;
- Возможность доставки статических файлов.

Для управления маршрутизацией используются роутеры, в которых происходит описание маршрутов. В каждом маршруте указывается путь и список middleware, то есть функций, обрабатывающих конкретный запрос.

После создания роутера его следует подключить как middleware в само приложение с помощью `app.use` где `app` является объектом `express` приложения. При подключении указывается путь, по которому будет располагаться все описанные в роутере маршруты и ссылка на объект роутера.

Таким образом `express.js` сильно упрощает процесс разработки серверной части системы.

Перед началом работы с данным фреймворком следует произвести установку среды выполнения кода – `node.js`.

Инициализация `node.js` проекта происходит с помощью команды `npm init`. После выполнения данной команды будет предложен ряд настроек, например, указать имя проекта, его версию, точку входа, то есть название файла запуска.

После инициализации проекта остается установить нужные библиотеки. Для начала следует установить `express.js` командой `npm install express --save`.

3.2.1 Библиотеки

Помимо библиотеки `express`, в проекте используется множество других библиотек.

Разбор принципа генерации JWT был произведен в параграфе 2.7. Но в проекте используется библиотека `jsonwebtoken`. Данная библиотека реализует функционал генерации и валидации веб токенов [7]. Экспорт данной библиотеки возвращает объект, предоставляющий два основных метода: `sign` и `verify`. Как можно понять из названий метод `sign` служит для создания токенов. Для генерации требуется передать данные, которые будут храниться в токене, и секретный ключ, которым может быть любая строка. Также

опционально можно передать время жизни токена и алгоритм хеширования для создания подписи. Метод `verify` служит для декодирования содержимого токена и подтверждения его валидности. Данный метод принимает два обязательных параметра это токен и ключ, который использовался для подписи токена. Также требуется передать название алгоритма хеширования если используется отличный алгоритм от того, что используется по умолчанию.

Так как создаваемое приложение является клиент-серверным то никто не исключает возможности установки веб-сервера, доставляющего статические файлы и сервера, предоставляющего API на разные домены, порты, IP адреса. И в таком случае при попытке отправить запрос на API с отличным, протоколом, портом или доменом мы можем получить ошибку `Cross-Origin Request Blocked`. Для решения этой проблемы используется модуль `cors`, который позволяет включить поддержку CORS (Cross-Origin Resource Sharing) в приложении [8].

Для подключения к базе данных и отправки запросов используется модуль `pgpromise`. Данный модуль предоставляет множество методов для отправки запросов к базе данных, а также осуществляет поддержку промисов. Основные методы:

- `db.any(query, [values])`: Выполняет запрос и возвращает все строки результата;
- `db.one(query, [values])`: Выполняет запрос и возвращает одну строку результата. Если строк больше или меньше, чем одна, выбрасывает ошибку;
- `db.none(query, [values])`: Выполняет запрос, который не должен возвращать строки (например, `INSERT` или `UPDATE`);
- `db.oneOrNone(query, [values])`: Выполняет запрос и возвращает одну строку результата или `null`, если строк нет;

- `db.many(query, [values])`: Выполняет запрос и возвращает все строки результата. Если строк нет, выбрасывает ошибку;
- `db.manyOrNone(query, [values])`: Выполняет запрос и возвращает все строки результата или пустой массив, если строк нет.

Данные для авторизации отправляются по электронной почте. NodeMailer предоставляет удобный способ отправки электронных сообщений по средствам smtp протокола [6]. После установки модуля требуется импортировать его и создать транспортер, для этого требуется вызвать метод `createTransport` и передать нужные для подключения к почтовому серверу данные. Этими данными являются имя хоста, порт и данные для авторизации. Если переданы верные данные, то можно приступить к отправке сообщений методом транспортера `sendMail`.

Пароли для авторизации генерируются случайным образом с помощью модуля `passfather`, который предоставляет возможность генерировать случайные строки, которые можно использовать в качестве пароля [9]. После установки импортирования представляет собой функцию, генерирующую пароли. По умолчанию генерирует пароли длиной 12 символов включающие в себя цифры, буквы латинского алфавита в верхнем и нижнем регистре и дополнительные символы. Каждый из типов символов можно отключить, длину также можно изменить. Для этого в функцию `passfather` требуется передать объект с нужными полями и их значениями.

Модуль `bcrypt` предоставляет возможность хеширования паролей [10]. После установки и импорта объект предоставляет два основных метода `hash` и `compare`. Метод `hash` создает хэш и содержит два обязательных параметра. Первый параметр принимает в себя данные, которые требуется хэшировать, а второй принимает число раундов при генерации соли или саму соль в виде строки. Соль — это особая строка, добавляемая к паролю для повышения безопасности создаваемых хэшей. Для сравнения пароля и его хэша

используется метод `compare` для использования требуется передать пароль и хешированную строку из базы данных. Метод возвращает булево значение означающее совпадение если метод вернул `true`, в случае несовпадения возвращает `false`.

А `cookie-parser` читает `Cookie` заголовок из запросов и сохраняет куки в поле `req.cookies` [11].

3.2.2 Файл настроек

Для хранения некоторых конфиденциальных и настроечных данных используется файл `«.env»`. Данный файл состоит из пар ключ-значение, где каждая пара записывается в новой строке. Пример хранения данных этом файле находится в листинге 1.

Листинг 1 – Пример данных хранящихся в `«.env»`

```
DATABASE_URL=postgres://user:password@localhost:5432/mydatabase
SECRET_KEY=mysecretkey
DEBUG=true
```

Для использования данных из данного файла также требуется модуль под названием `dotenv`. После установки данного модуля требуется его импортировать и вызвать метод `config`. После вызова данного метода доступ к переменным хранящимся в файле `«.env»` можно будет получить в любом файле приложения через `process.env`. Формат хранящихся в файле данных показан в листинге 2.

Листинг 2 – Формат данных хранящихся в файле `«.env»`

```
PORT = <ПОРТ НА КОТОРОМ БУДЕТ ЗАПУЩЕН СЕРВЕР>
DATABASE_IP = <IP ХОСТА НА КОТОРОМ ЗАПУЩЕНА БАЗА ДАННЫХ>
DATABASE_PORT = <ПОРТ НА КОТОРОМ ЗАПУЩЕНА БАЗА ДАННЫХ>
DATABASE_USERNAME = <ИМЯ ПОЛЬЗОВАТЕЛЯ ЧЕРЕЗ КОТОРОГО БУДЕТ ПРОИЗВОДИТЬСЯ
ПОДКЛЮЧЕНИЕ К БАЗЕ ДАННЫХ>
DATABASE_PASSWORD = <ПАРОЛЬ ПОЛЬЗОВАТЕЛЯ ЧЕРЕЗ КОТОРОГО БУДЕТ ПРОИЗВОДИТЬСЯ
ПОДКЛЮЧЕНИЕ К БАЗЕ ДАННЫХ>
DATABASE_NAME = <ИМЯ БАЗЫ ДАННЫХ ЧЕРЕЗ КОТОРОЕ БУДЕТ ПРОИСХОДИТЬ ПОДКЛЮЧЕНИЕ>
JWT_SECRET_KEY = <КОД ИСПОЛЬЗУЕМЫЙ ПРИ ГЕНЕРАЦИИ ACCESS ТОКЕНОВ>
REFRESH_SECRET_KEY = <КОД ИСПОЛЬЗУЕМЫЙ ПРИ ГЕНЕРАЦИИ REFRESH ТОКЕНОВ>
JWT_TOKEN_EXPIRES = <ВРЕМЯ ЖИЗНИ ACCESS ТОКЕНОВ>
REFRESH_TOKEN_EXPIRES = <ВРЕМЯ ЖИЗНИ REFRESH ТОКЕНОВ>
CLIENT_URL = <АДРЕС ХОСТА С КОТОРОГО ПРОИСХОДИТ ДОСТАВКА СТАТИЧЕСКИХ ФАЙЛОВ>
MAIL_HOST = <АДРЕС ХОСТА НА КОТОРОМ НАХОДИТСЯ ПОЧТОВЫЙ СЕРВЕР>
MAIL_PORT = <ПОРТ НА КОТОРОМ НАХОДИТСЯ ПОЧТОВЫЙ СЕРВЕР>
MAIL_USER = <ИМЯ ПОЛЬЗОВАТЕЛЯ ИСПОЛЬЗУЕМОЕ ПРИ АВТРОИЗАЦИИ>
MAIL_PASSWORD = <ПАРОЛЬ ИСПОЛЬЗУЕМЫЙ ПРИ АВТРОИЗАЦИИ>
```

Таким образом происходит хранение данных настроек в создаваемом проекте.

3.2.3 Взаимодействие с базой

Взаимодействие с базой данных происходит по средствам модуля `pgpromise`. Данный модуль предоставляет возможность для удобного взаимодействия с базой данных с помощью функций поддерживающих промисы [13]. Для начала работы с данным модулем требуется создать объект подключения, через который будет происходить отправка всех последующих запросов. Создание такого объекта вынесено в отдельный файл `dbContext.js`. Код данного файла показан в листинге 3.

Листинг 3 – Код файла `dbContext.js`

```
const pgPromise = require("pg-promise");

const pgp = pgPromise();

const DATABASE_USERNAME = process.env.DATABASE_USERNAME;
const DATABASE_PASSWORD = process.env.DATABASE_PASSWORD;
const DATABASE_IP = process.env.DATABASE_IP;
const DATABASE_PORT = process.env.DATABASE_PORT;
const DATABASE_NAME = process.env.DATABASE_NAME;

const cn =
`postgres://${DATABASE_USERNAME}:${DATABASE_PASSWORD}@${DATABASE_IP}:${DATABASE_PORT}/${DATABASE_NAME}`;
const db = pgp(cn);

module.exports = db;
```

В данном файле происходит импорт модуля `pgpromise`. Далее происходит инициализация модуля без передачи опций инициализации и создание объекта базы данных путем передачи строки подключения в функцию `pgp`. Формирования строки подключения происходит по шаблону, в который предаются данные взятые из файла «.env».

Далее в файл `dbService.js` происходит импорт объекта базы данных и с его помощью производятся запросы. Методы, используемые для отправки запросов описаны в параграфе 3.2.1. Пример описанных в файле `dbService.js` запросов показан в листинге 4.

Листинг 4 – Пример описания запросов

```
async getNewNotificationsNoId(recipientId, lastId){
    return db.manyOrNone({
        name: 'getNewNotificationsNoId',
        text: 'SELECT n.id_notification, n.id_notification_type,
n.notification_text, n.notification_header, sn.id_sended, sn.id_sender,
sn.is_reacted, sn.time FROM system.notifications AS n JOIN
system.sended_notifications AS sn ON n.id_notification = sn.id_notification
WHERE sn.id_recipient = $1 AND sn.id_sended > $2 ORDER BY n.id_notification
DESC LIMIT 10;',
        values: [recipientId, lastId]
    });
}
async getCredentials(user_name){
    return db.oneOrNone({
        name: 'getCredInfoByUserName',
        text: 'SELECT password, id_employee, valid, roles FROM
system.credentials WHERE user_name = $1 AND valid = true',
        values: [user_name]
    });
}
```

Эти запросы описаны в классе `dbService` далее его можно импортировать в нужный файл. При импорте будет создаваться объект данного класса, а в полученном объекте будут доступны все запросы в виде методов.

3.2.4 Принцип обработки запросов

Основой приложения на `express.js` являются маршруты(`router`) и промежуточные функции(`middleware`). Маршруты позволяют определить как приложение будет реагировать на забросы по тем или иным путям. К каждому из путей будет привязан список из промежуточных функций, занимающихся обработкой запроса.

Начальная настройка приложения происходит в файле `index.js`. Код данного файла представлен в приложении В.

В данном файле происходит импорт необходимых модулей их настройка в качестве промежуточных функций при помощи `app.use`. Промежуточные функции определяют последовательность обработки запросов. Таким образом при поступлении запроса будет происходить следующее:

- Извлечение информации из тела запроса;
- Извлечение информации из `cookie`;
- Запись необходимых заголовков в ответ;

- Проверка маршрута (переход в роутер в случае совпадения);
- Переход к последней промежуточной функции происходит в случае ошибки. Функция `errorMiddleware` является обработчиком ошибок.

В роутерах также происходит проверка пути, по которому был отправлен запрос. В случае совпадения происходит обработка запроса в переданных в путь промежуточных функциях. Код одного из роутеров представлен в листинге 5.

Листинг 5 – Код файла `authRouter.js`

```
const Router = require('express').Router;
const authController = require('../controllers/authController');
const {body} = require('express-validator');
const authMiddleware = require('../middlewares/authMiddleware');
const rolesMiddleware = require('../middlewares/rolesMiddleware');

const router = new Router();

router.post('/login',
  body('login').isLength({min: 3, max: 50}),
  body('password').isLength({min: 3, max: 50}),
  authController.login
)
router.post('/register', authMiddleware, (req, res, next)=>
{res.locals.requiredRoles = ["register", "admin"]; next()}, rolesMiddleware,
authController.register)
router.get('/logout', authController.logout)
router.get('/refresh', authController.refresh)
router.post('/getUsersData', authMiddleware, (req, res, next)=>
{res.locals.requiredRoles = ["register", "admin"]; next()}, rolesMiddleware,
authController.getUsersData);
router.post('/updateRecipientsInfo', authMiddleware, (req, res, next)=>
{res.locals.requiredRoles = ["register", "admin"]; next()}, rolesMiddleware,
authController.updateRecipientsInfo);
//router.get('/user', authMiddleware, authController.user)

module.exports = router;
```

Переданные в пути промежуточные функции также определяют последовательность обработки запросов. `AuthMiddleware` и `rolesMiddleware` проверяют авторизацию пользователя и наличие требуемых ролей для доступа к определённой информации. Последним в последовательности находятся обработчики, описанные в `authController.js` и `notificationController.js`.

Обработчиком также является промежуточная функция. Промежуточные функции должны принимать три параметра: `req` – запрос, `res`

– ответ, next – функция с помощью которой происходит передача управления к следующей промежуточной функции или обработчику маршрута. Пример такого обработчика представлен в листинге 6.

Листинг 6 – Обработчик login

```
async login(req, res, next){
  try{
    const {login, password} = req.body;
    const payload = await authService.login(login, password);
    res.cookie('refreshToken', payload.tokens.refreshToken, {maxAge:
process.env.REFRESH_TOKEN_EXPIRES * 24 * 60 * 60 * 1000, httpOnly: true})
    return res.json({accessToken: payload.tokens.accessToken,
userData: payload.userData, roles: payload.roles});
  } catch(e) {
    next(e);
  }
}
```

В обработчиках происходят обращения к сервисам, производящим какие-либо действия. Всего в проекте представлено пять сервисов:

- authService – производит действия связанные с авторизацией.
- notificationService – производит действия связанные с уведомлениями.
- dbService – производит запросы к базе данных.
- mailService – производит рассылку сообщений по электронной почте.
- tokenService – производит генерацию и валидацию токенов.

3.2.5 Промежуточные функции

В проекте помимо обработчиков представлены еще три промежуточные функции: authMiddleware, errorMiddleware, rolesMiddleware.

Начнем с authMiddleware в нем происходит проверка валидности access токена, получение данных хранящихся в токене и сохранение этих данных в res.local.user. В случае возникновения ошибки происходит генерация ошибки Unauthorized. Код данной функции представлен в листинге 7.

Листинг 7 – Код функции authMiddleware

```
const ApiError = require('../exceptions/apiError');
const tokenService = require('../services/tokenService');
```

Продолжение листинга 7

```
module.exports = function(req, res, next){
  try{
    const authorizationHeader = req.headers.authorization;
    if(!authorizationHeader){
      return next(ApiError.UnauthorizedError());
    }

    const accessToken = authorizationHeader.split(' ')[1];

    const userData = tokenService.validateAccessToken(accessToken);
    if(!userData){
      return next(ApiError.UnauthorizedError());
    }
    res.locals.user = userData;
    //console.log(userData);
    next();
  } catch(e) {
    return next(ApiError.UnauthorizedError());
  }
}
```

`ErrorMiddleware` проверяет тип поступившей в него ошибки. В случае если поступившая ошибка имеет класс `ApiError`, то происходит генерация ответа на основе находившихся в объекте этого класса данных. А в случае, если ошибка имеет другой класс, то происходит генерация ответа с кодом 500 и сообщением «Непредвиденная ошибка».

`RolesMiddleware` производит проверку наличия необходимых ролей у пользователя отправившего запрос. Перед ним должны быть выполнены `authMiddleware` и еще одна промежуточная функция в виде лямбда выражения, в которой указываются необходимые роли. В случае отсутствия необходимых ролей происходит генерация ошибки `AccessDenied`.

3.2.6 Аутентификация и регистрация

Аутентификация в системе происходит по средствам ввода логина и пароля. Введенные логин и пароль отправляются на сервер по пути `auth/login`. В `auth` роутере описаны все маршруты касающиеся аутентификации, авторизации, регистрации и изменения данных пользователей. Всего в нем описано 6 путей. Перечень путей с их описанием представлен в таблице 3.

Таблица 3 – Пути используемые при аутентификации и регистрации

Путь	Принимает	Возвращает	Описание
login	Json содержащий логи и пароль	Cookie с refresh токеном и Json объект с access токеном	Используется для аутентификации пользователей
register	Json с данными сотрудника	Статус запроса	Используется для регистрации пользователей
logout	Refresh токен из cookie	Статус запроса	Используется для удаления refresh токена при выходе из приложения
refresh	Refresh токен из cookie	Refresh и access токены	Используется для обновления пары токенов
getUsersData	Id последнего полученного пользователя	Данные пользователей id которых меньше полученного	Для получения данных пользователей на странице регистрации
updateRecipientsInfo	Данные пользователей	Статус запроса	Обновление данных пользователей на странице регистрации

С помощью данных маршрутов происходит авторизация, аутентификация и регистрация пользователей.

3.2.7 Оповещения

Обработка данных касающихся уведомлений происходит в роутере находящемся по пути /notification. Данный роутер и все вложенные в него пути описаны в файле notificationRouter.js. Каждый из путей доступен только авторизованным пользователям. Всего в нем описано 16 путей. Перечень путей с их описанием представлен в таблице 4.

Таблица 4 – Пути используемы при управлении уведомлениями

Путь	Принимает	Возвращает	Описание
types	Запрос	Id и названия типов оповещений	Используется для получения типов оповещений
recipients	Запрос	ФИО и id зарегистрированных пользователей.	Используется для получения данных пользователей системы для отображения их данных
positions	Запрос	Id и названия должностей	Используется для получения данных о должностях
departments	Запрос	Id и названия отделов	Используется для получения данных о должностях
lastId	Запрос	Id последнего полученного пользователем уведомления	Используется для инициализации данных об уведомлениях
get	Id последнего загруженного уведомления	Десять уведомлений, относящихся к пользователю и имеющих меньший id	Используется для загрузки уведомлений
getNew	Id последнего полученного пользователем уведомления	Уведомления, относящиеся к пользователю id которых больше полученного в запросе	Используется для получения новых уведомлений
send	Данные уведомления	Статус запроса	Используется для отправки уведомлений

Продолжение таблицы 4

react	Id оповещения к которому привязана реакция, id оповещения которое будет отправлено	Статус запроса	Используется для отправки реакций на уведомления
unreactedCount	Запрос	Количество оповещений, требующих реакции в том или ином типе	Используется для загрузки количества уведомлений, требующих реакции
getSendedNotifications	Id последнего загруженного оповещения	Двадцать оповещений, следующих после него	Используется для получения данных об отправленных уведомлениях
getComments	Id оповещения	Комментарии к оповещению	Используется для получения комментариев
getCommentsCount	Список из id оповещений количество комментариев, к которым требуется загрузить	Количество комментариев для каждого из оповещений	Используется для получения информации о количестве комментариев под оповещением
getNewComments	Список из id оповещений для обновления комментариев	Новые комментарии к оповещению	Используется для обновления комментариев
newComment	Id оповещения и текст комментария	Статус запроса	Используется для отправки комментариев
updateSended	Список из id оповещений	Данные оповещений	Используется для обновления данных об отправленных оповещениях

3.3 Браузерная часть

Клиентская часть приложения реализована с использованием фреймворка vue.js. Данный фреймворк используется для создания пользовательских интерфейсов и предоставляет удобный способ описания в виде компонентов [16].

Основными особенностями vue.js являются:

- Компонентный подход. Данный фреймворк позволяет делить приложение на независимые, повторно используемые компоненты. Каждый такой компонент может содержать свою логику, шаблоны и стили;
- Система шаблонов. Vue.js предоставляет возможность использования HTML подобных шаблонов, позволяющих легко описывать пользовательский интерфейс;
- Директивы. Фреймворк предоставляет набор директив позволяющих связывать данные компонента с DOM-элементами;
- Реактивность. В фреймворке отрисовка элементов происходит на основе реактивной системы данных, позволяющей автоматически обновлять элементы пользовательского интерфейса при изменении данных.

Основным элементом фреймворка является компонент. Компоненты описываются в файлах формата «.vue». Они состоят из 3 основных элементов: template – описание HTML шаблона, script – описание данных и логики работы компонента, style – описание CSS стилей, применяемых к компоненту. Пример компонента приведен в листинге 8.

Листинг 8 – Код функции errorMiddleware

```
<template>
  <div>
    <h1>{{ message }}</h1>
    <button @click="updateMessage">Click me</button>
  </div>
</template>
<script>
```

Продолжение листинга 8

```
export default {
  data() {
    return {
      message: 'Hello, Vue.js!'
    };
  },
  methods: {
    updateMessage() {
      this.message = 'You clicked the button!';
    }
  }
};
</script>
<style scoped>
h1 {
  color: blue;
}
</style>
```

Нажатие на кнопку в примере выше привязано к функции `updateMessage`, выполнение которой изменяет поле `message`. Благодаря реактивности обновленная строка будет выведена на экран сразу после ее изменения.

В проекте используются некоторые директивы. Их список с описанием приведен далее:

- `v-bind`. Данная директива позволяет связывать данные `vue` с атрибутами HTML элементов;
- `v-model`. Позволяет производить двустороннее связывание данных из элементов ввода с данными `vue`;
- `v-if`. Позволяет контролировать добавление элемента в DOM по условию;
- `v-for`. Позволяет отображать данные из списков по шаблону;
- `v-show`. Похожа на `v-if`, но в отличие от нее элемент с этой директивой всегда находится в DOM. В случае неудовлетворения условия скрывается CSS свойством `display: none`.

3.3.1 Библиотеки

В проекте используются дополнительные библиотеки: `vuex`, `vue route`, `axios`.

Vuex – библиотека предоставляющая возможность создания централизованного хранилища доступ к которому будет доступен в любом компоненте [14]. Хранилище поделено на 4 части:

- State или состояние. Это объект, содержащий в себе все переменные хранилища;
- Getters. Это вычисляемые свойства позволяющие извлекать и обрабатывать хранящиеся в состоянии данные;
- Mutations или мутации. Представляют собой функции, изменяющие состояние;
- Actions. Асинхронные функции, имеющие доступ к Getters и Mutations. Позволяют реализовать удобный интерфейс для взаимодействия с API.

Пример хранилища представлен в приложении Г.

Vue router – это официальная библиотека предоставляющая возможность добавления навигации по различным маршрутам для одностраничного приложения [15]. После установки данной библиотеки требуется создание и настройка роутера. Настройка роутера происходит путем связывания компонентов и путей. Пример настроенного роутера показан в листинге 9.

Листинг 9 – Код роутера

```
const routes = [  
  {  
    path: '/',  
    component: loginPage,  
    name: 'loginPage'  
  },  
  {  
    path: '/app',  
    component: mainPage,  
    name: 'mainPage',  
  },  
  {  
    path: '/register',  
    component: registerPage,  
    name: 'register'  
  },  
  {
```

Продолжение листинга 9

```
    path: '/sendedNotifications',
    component: sendedNotificationsPage,
    name: 'sendedNotificationsPage'
  },
  {
    path: '/*:catchAll(.*)',
    component: notFoundPage,
    name: 'notFoundPage'
  },
]
export default routes
```

После создания роутера его следует подключить в виде плагина с помощью `app.use`. Также дается возможность подключения хуков к роутерам. Хуки позволяют более подробно описать логику переходов между различными путями. Пример хука показан в листинге 10.

Листинг 10 – Код хука

```
router.beforeEach(async (to, from, next) => {
  await store.dispatch('user/checkAuth');
  if(store.getters['user/getAuthState'] == true){
    await store.dispatch('positions/loadPositions');
    await store.dispatch('employees/loadEmployees');
    await store.dispatch('departments/loadDepartments');
    await store.dispatch('notificationTypes/loadNotificationTypes');
    if(to.name == 'loginPage'){
      next({ name: 'mainPage' });
    }else{
      next();
    }
    to;
    from;
  }else{
    if(to.name != 'loginPage'){
      next({ name: 'loginPage' });
    }else{
      next();
    }
  }
});
```

В хуке, продемонстрированном в листинге 10 описана логика, не позволяющая авторизованным пользователям перейти на страницу входа, а не авторизованным перейти на страницу отличную от страницы входа.

Для взаимодействия с API используется библиотека `axios` предоставляющая расширенные возможности для описания запросов такие как: инстансы и интерцепторы [3].

Инстансы позволяют создавать настраиваемые экземпляры axios. Они используются для в случае необходимости отправки множества запросов с одинаковыми настройками такими как: базовая ссылка, заголовки запроса или задержка.

Интерцепторы позволяют перехватывать запросы и менять их в зависимости от их состояния. Их можно крепить к инстансам что позволяет настроить логику обработки сразу для множества запросов. Пример настроенного инстанса показан в листинге 11.

Листинг 11 – Код инстанса axios

```
import axios from 'axios'
const instance = axios.create({
  baseURL: 'http://localhost:3000/',
  withCredentials: true,
  headers: {
    accept: 'application/json'
  }
})
instance.interceptors.request.use((config) => {
  config.headers.Authorization = `Bearer ${localStorage.getItem('token')}`;
  return config;
})

instance.interceptors.response.use((config) => {
  return config
}, async (error) => {
  const originalRequest = error.config;
  if(error.response.status === 401 && error.config && !error.config._isRetry){
    originalRequest._isRetry = true;
    try{
      const response = await axios.get('http://localhost:3000/auth/refresh',
{withCredentials: true});
      localStorage.setItem('token', response.data.accessToken);
      return instance.request(originalRequest);
    } catch(e){
      console.log('Не авторизован');
    }
  }
  throw error;
})
export default instance
```

В коде листинга 11 происходит настройка инстанса через который происходит обработка всех api запросов клиентской части приложения. К инстансу прикреплены два интерцептора: первый записывает access token в заголовок запроса, второй при возникновении ошибки 401 производит попытку обновления пары токенов.

3.3.2 Архитектура

Клиентскую часть приложения можно поделить на три основные части: компоненты, хранилище и API. Связи между частями продемонстрированы на рисунке 26.



Рисунок 26 – Связи между частями приложения

Далее рассмотрим каждую из частей более подробно. В компонентах описывается интерфейс и логика его работы. Схема вложенности компонентов продемонстрирована на рисунке 27.

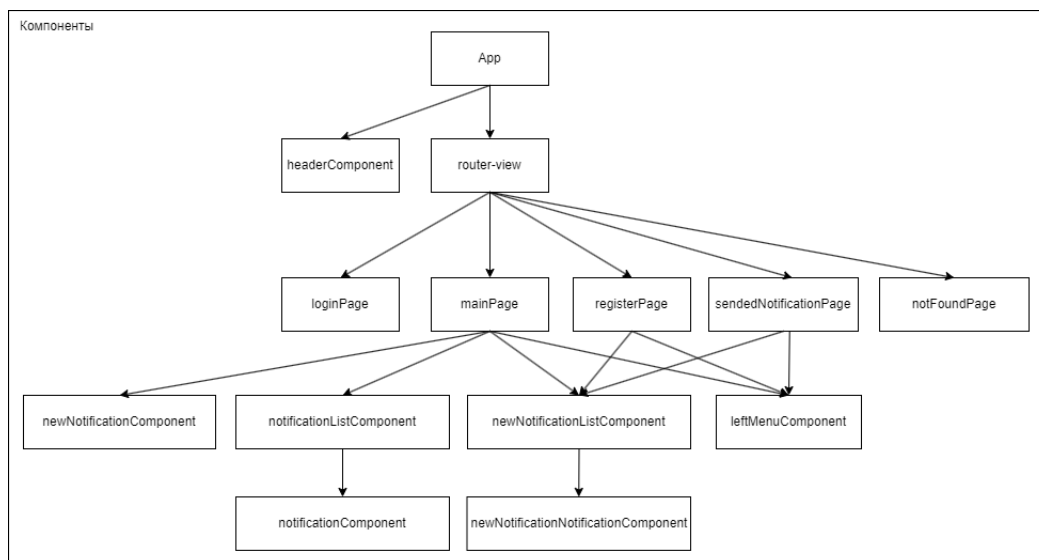


Рисунок 27 – Схема вложенности компонентов

В хранилище происходит обработка и хранение данных. Также оно поделено на десять частей, каждая из которых хранит свой набор данных. Схема хранилища представлена на рисунке 28.

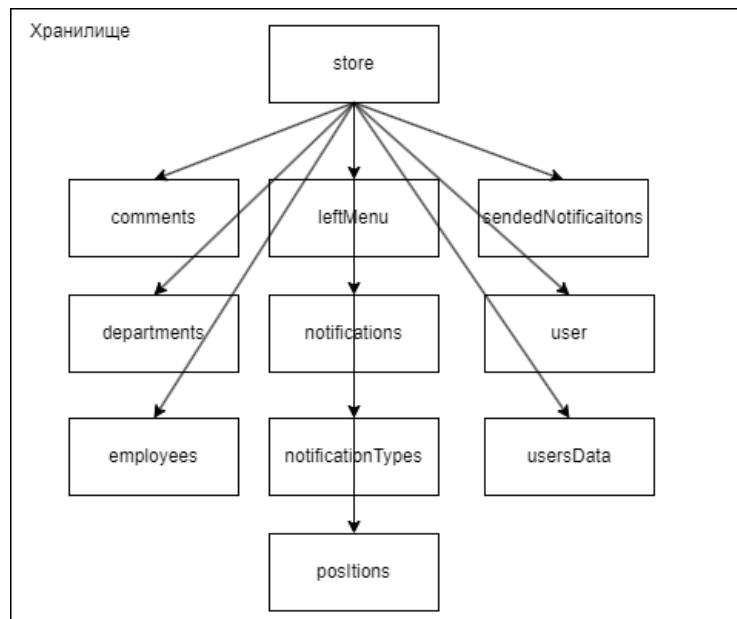


Рисунок 28 – Схема хранилища

Схема части, отвечающей за API запросы показана на рисунке 29.

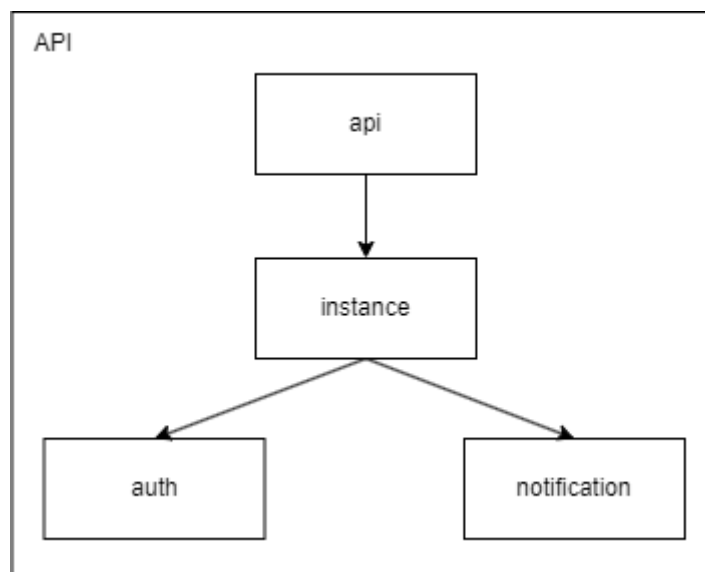


Рисунок 29 – Схема API

Таким образом была выстроена клиентская часть системы оповещения.

3.3.3 Аутентификация

Страница аутентификации описана в одном компоненте под названием loginPage. Код данного компонента представлен в приложении Д.

В шаблоне компонента описана форма ввода логина и пароля, состоящая из двух полей ввода и кнопки. Информация из полей ввода связана с

переменными vue по средствам директива v-model. Также информация из данных полей валидируется с помощью vuelidate. По нажатию на кнопку выполняется метод logInto, который проверяет наличие ошибок валидации. При их отсутствии выполняется action login из хранилища user, который отправляет введенные данные на сервер и в случае успешной авторизации возвращает данные пользователя и пару access и refresh токенов. Полученные данные сохраняются в хранилище user и происходит перенаправление на страницу mainPage. Вид интерфейса продемонстрирован на рисунке 30.

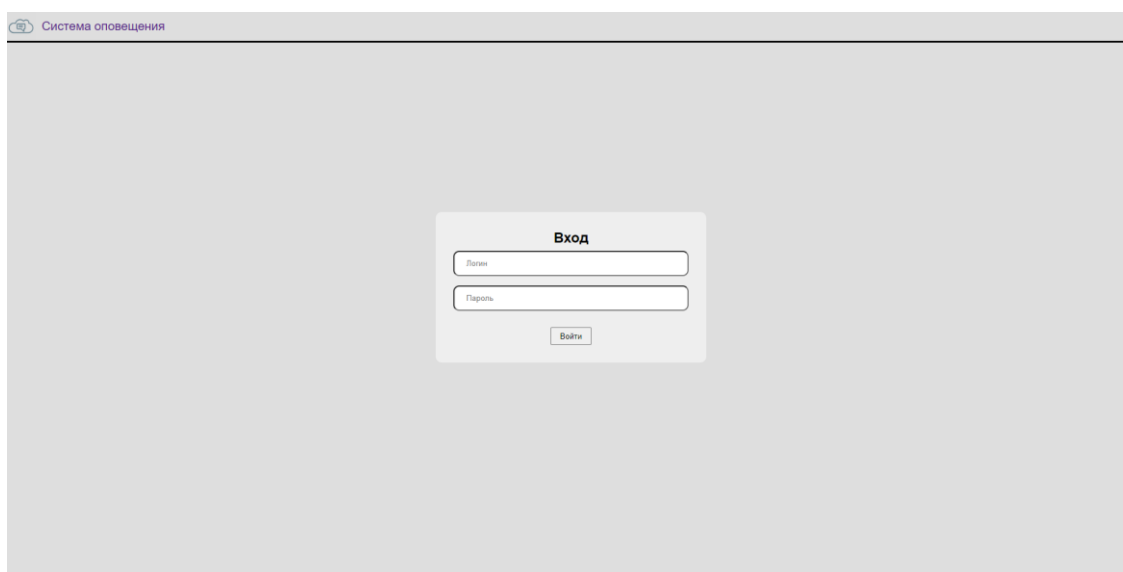


Рисунок 30 – Интерфейс страницы авторизации

Таким была реализована страница авторизации в системе оповещения сотрудников компании.

3.3.4 Уведомления

Просмотр и взаимодействие с уведомлениями в основном происходит на главной странице приложения. Данная страница описана в компоненте под названием mainPage. Код этого компонента представлен в приложении Е.

Компоненты в данном проекте логически поделены на два типа: компоненты и страницы. Компоненты являются частями страниц, а страницы привязываются к путям и отображаются через router-view. В приложении Е продемонстрирована страница, состоящая из четырех компонентов:

leftMenuComponent, newNotificationComponent, notificationListComponent, newNotificationsListComponent. Эти компоненты импортируются и расставляются в шаблоне. Также в этом компоненте хранится информация о типе отображаемых уведомлений, о том открыто ли модальное окно для создания уведомлений и таймер, запускающий функцию, отправляющую запросы на обновление данных. За запуском и остановкой таймера следят хуки жизненного цикла компонента. Хук `created` выполняется после создания компонента и запускает таймер, а `beforeUnmount` запускается перед уничтожением компонента и очищает таймер.

Интерфейс главной страницы продемонстрирован на рисунке 31.

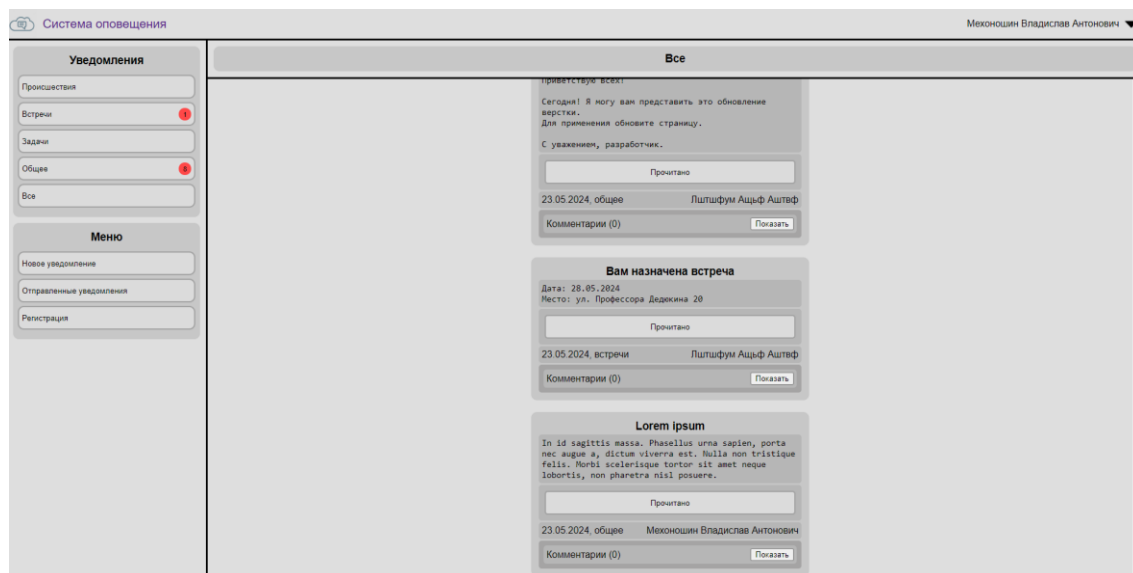


Рисунок 31 – Интерфейс главной страницы приложения

В следующих четырех параграфах будет описано устройство компонентов, использованных на странице `mainPage`.

3.3.4.1 Меню

За переключение типов отображаемых уведомлений и переходы между страницами приложения отвечает компонент под названием `leftMenuComponent`. Код данного компонента представлен в приложении Ё.

Данный компонент представляет из себя два списка кнопок. Первый список формируется из содержащихся в хранилище `notificationTypes` типов

уведомлений с помощью директивы `v-for`. Второй список уже заранее сформирован, но может изменяться в зависимости от страницы, на которой находится пользователь или ролей пользователя. Например, кнопка «Новое уведомление» доступна только на главной странице, а кнопка «регистрация» пользователям с ролью `register` или `admin`. При переходе приложения в мобильный режим компонент может скрываться. Передача данных из дочернего компонента в родительский осуществляется по средствам вызова события с определенным именем в дочернем компоненте и привязкой методов к данным событиям в родительском компоненте.

3.3.4.2 Список оповещений

Список оповещений описан в компоненте `notificationListComponent` и отображается только на главной странице. Код компонента представлен в приложении Ж.

Список состоит из компонентов `notificationComponent` формируемых директивой `v-for`. Перед созданием компонента выполняются запросы на получение оповещений и количества комментариев под ними. Оповещения подгружаются по десять штук. При достижении конца списка выполняется запрос на загрузку еще десяти оповещений. Проверка достижения конца списка описана в методе `listScroll`. Данный метод привязан к событию `scroll` в DOM и выполняется при изменении положения ползунка прокрутки. Компонент `notificationComponent` принимает информацию о компонентах в виде пропов и отображает её.

3.3.4.3 Уведомления о новых оповещениях

Уведомления о новых оповещениях отображаются в компоненте `newNotificationListComponent`, он отображается на всех страницах приложения кроме страницы `loginPage`. Код данного компонента представлен в приложении З.

Данный компонент отображает данные из хранилища `notifications` с помощью директивы `v-for`. Уведомления исчезают из данного списка либо по

нажатию на крестик в правой их части, либо по нажатию на кнопку реакции. В `newNotificationNotificationComponent` определено событие, к которому привязан метод `closeModal` удаляющий уведомление из списка новых уведомлений по его `id`.

3.3.4.4 Создание оповещений

Создание оповещений доступно только на главной странице приложения. Панель для создания оповещений открывается по нажатию кнопки «Новое уведомление» в левом меню. Сама панель создания уведомлений описана в компоненте `newNotificationComponent`. Код данного компонента представлен в приложении И.

Форма создания оповещений состоит из нескольких частей позволяющих выбрать тип уведомления, ввести текст заголовка уведомления и текст самого уведомления. А также выбрать получателей, разделенных на группы по отделам и должностям и настроить реакции. Настройка реакций состоит из ввода имени кнопки, по которой будет производиться и настройки уведомления, которое будет отправлено по нажатию на кнопку реакции получателем уведомления.

Панель с получателями формируется на основе объекта формируемого `computed` свойством `departmentData`. Использование `computed` свойства позволяет кешировать и повторно использовать созданный им объект, пересчитан он будет только при изменении данных, на основе которых был создан объект.

По нажатию на кнопку «Отправить» будет выполнен метод `sendNotification`, который составит из данных формы один объект и произведет его отправку на сервер. Форма создания уведомлений представлена на двух рисунках 32 и 33.

Рисунок 32 – Интерфейс формы создания уведомлений

Рисунок 33 – Интерфейс настройки реакций

Таким образом была реализована форма создания уведомлений.

3.4.4 Отправленные оповещения

Просмотр информации об отправленных уведомлениях производится на странице описной в компоненте `sendedNotificationsPage`. Код данного компонента представлен в приложении Й.

Данный компонент позволяет просмотреть, когда, кому и какие уведомления были отправлены, а также информацию о том, прореагировали ли получатели на уведомления и если прореагировали, то через сколько минут

с момента отправки. В дополнение к этому предоставляется возможность просмотра и отправки комментариев к уведомлениям.

При переходе на страницу выполняется запрос на получение данных об отправленных уведомлениях и инициализация таймера, выполняющего запросы на обновление данных на данной странице.

Интерфейс данной страницы представлен на рисунке 34.



Рисунок 34 – Интерфейс просмотра информации об отправленных уведомлениях

3.4.5 Регистрация

Регистрация пользователей системы и редактирование их данных производится работниками отдела кадров на странице registerPage. Код компонента данной страницы представлен в приложении К.

Страница состоит из двух форм: форма регистрации новых сотрудников и форма изменения данных зарегистрированных сотрудников.

Регистрация происходит путем ввода данных сотрудника и нажатия на кнопку «Отправить». После успешной регистрации на указанную электронную почту должны будут отправлены данные для авторизации в системе.

Форма изменения данных представляет собой список пользователей, отсортированный по дате регистрации. После изменения данных в поле пользователя, объект с данными пользователя помещается в отдельный массив с объектами, в которых произошли изменения. По нажатию на кнопку «Отправить обновления» происходит отправка массива измененных объектов на сервер и последующая запись, а базу данных. После получения ответа на данный запрос происходит очистка массива с измененными объектами и запрос обновленных данных пользователей. Интерфейс страницы регистрации представлен на рисунке 35.

Система оповещения

Лилифун Адаф Ашриф

Меню

- На главную страницу
- Отправленные уведомления
- Регистрация

Регистрация новых сотрудников

Отдел:

Должность:

Имя:

Фамилия:

Отчество:

Электронная почта:

Отправить

Изменение данных

Обновить

Отправить обновления

Имя	Имя1
Фамилия	Фамилия1
Отчество	Отчество1
Отдел	Текст
Должность	Благодарен
Роль	user
Зарегистрирован	true
Работает	false
Имя	Имя2
Фамилия	Фамилия2
Отчество	Отчество2
Отдел	Текст
Должность	Благодарен

Рисунок 35 – Интерфейс просмотра информации об отправленных уведомлениях

3.5 Функциональное тестирование

Целью тестирования является установление соответствия функциональным требованиям разработанной системы. Тестирование проводится на основе технического задания демонстрируемого в приложении А методом черного ящика. Результат тестирования показан в таблице 5.

Таблица 5 – Порядок тестирования

Требование	Порядок тестирования	Результат тестирования
Аутентификация	1. Перейти на страницу приложения 2. На отобразившейся странице ввести неверные данные аутентификации и отправить 3. Ввести верные данные аутентификации и отправить	1. Пользователь успешно перешел на страницу приложения 2. После отправки неверных данных аутентификации была показана ошибка «Неверный логин или пароль» 3. После отправки верных данных пользователь был перенаправлен на главную страницу приложения
Регистрация и проверка прав доступа	1. Проверить наличие кнопки регистрации от имени пользователя, не имеющего нужных ролей 2. Проверить наличие кнопки от пользователя, имеющего требуемые роли 3. Ввести данные пользователя и отправить	1. Кнопки регистрации обнаружено не было 2. Кнопка отображалась по нажатию на нее был осуществлен переход на страницу регистрации 3. После ввода данных и отправки данных было отображено сообщение об успешной регистрации

Продолжение таблицы 5

Регистрация и проверка прав доступа	4. Войти в систему с полученными данными аутентификации 5. Проверить указанную электронную почту	4. При проверке электронной почты было обнаружено сообщение с данными аутентификации 5. С полученными данными был успешно произведен вход в систему
Гибкость	1. Добавить отдел, должность и тип уведомления и обновить страницу 2. Проверить наличие введенных данных	1. Были успешно добавлены отдел, должность и тип уведомления 2. После обновления страницы были обнаружены новый отдел, должность и тип уведомлений
Оповещение сотрудников	1. Отправить уведомление 2. Проверить наличие уведомления у получателей	1. Было отправлено уведомление двум пользователям 2. У обеих получателей было обнаружено уведомление о новом уведомлении.
Написание комментариев	1. Отправитель уведомления пишет комментарий под уведомлением 2. Получатель проверяет его наличие	1. На странице отправленных уведомлений отправителем был отправлен новый комментарий 2. Получателем был обнаружен новый комментарий

Продолжение таблицы 5

Привязка реакций	1. Отправить уведомление с двумя привязанными реакциями, при использовании которых должны быть отправлены разные уведомления отправителю. 2. Получатели должны отреагировать, используя разные реакции 3. Отправитель должен получить два разных уведомления	1. Было успешно отправлено уведомление с двумя привязанными реакциями 2. Получатели обнаружили кнопки реакции под полученным уведомлением и отреагировали нажатиями на разные кнопки 3. Отправитель получил два разных уведомления
Отслеживание осведомленности	1. Отправить уведомление 2. Перейти на страницу отправленных уведомлений 3. Просмотреть количество отреагировавших получателей 4. Отреагировать с аккаунта одного из получателей 5. Просмотреть обновлённые данные	1. Уведомление было успешно отправлено получателям 2. Был успешно совершен переход на страницу отправленных уведомлений 3. Количество отреагировавших пользователей было равно нулю 4. Один из получателей успешно отреагировал на уведомление 5. Количество отреагировавших пользователей было равно одному

Все описанные в таблице выше тесты были успешно пройдены. Таким образом, можно сказать, что разработанная система соответствует описанным в техническом задании функциональным требованиям.

3.6 Выводы по разделу

В данном разделе были описаны процесс и тонкости разработки системы оповещения сотрудников компании.

Был описан процесс разработки базы данных на основе СУБД PostgreSQL с использованием графической оболочки PgAdmin.

Был описан процесс создания серверной и клиентской частей приложения.

И наконец, было проведено функциональное тестирование приложения на основе требований из технического задания. Все пункты функционального тестирования были успешно выполнены, что говорит о соответствии системы функциональным требованиям из технического задания.

4 Экономический раздел

4.1 Техничко-экономическое обоснование проекта

На данный момент большая часть компаний использует различные мессенджеры для оповещения сотрудников. Целью создаваемой системы является повышение эффективности сотрудников компании за счет использования «реакций» и возможности массовой рассылки оповещений. В конечном итоге ожидается снижение времени на взаимодействие между сотрудниками до 50%.

4.2 Расчет экономического эффекта

Если допустить, что сотрудники в среднем тратят 1 час в день на взаимодействие между собой, а в году 200 рабочих дней, то при средней стоимости часа работы в 300р затраты на взаимодействие сотрудников составляют 60000р в год на человека.

При снижении времени взаимодействия на 50% получается экономия 30000р в год за каждого сотрудника.

Предположим, что в компании работает 50 человек. Таким образом компания сможет экономить 1500000р в год.

При стоимости программы в 500000р и всей требуемой инфраструктуры в 1000000р. Срок окупаемости составит 1,5года.

4.3 Выводы по разделу

В результате проведенного технико-экономического обоснования проекта внедрения системы для повышения эффективности взаимодействия сотрудников компании была подтверждена целесообразность ввода системы в эксплуатацию.

5 Защита информации

5.1 Определение класса защищённости

Система нацелена на предприятия государственного сектора, поэтому она подчиняется приказу ФСТЭК от 11.02.2013 «Об утверждении требований о защите информации, не составляющей государственную тайну, содержащейся в государственных информационных системах».

Система имеет объектовый масштаб, так как нацелена на использование в рамках одного предприятия

Для определения уровня значимости системы необходимо оценить возможный ущерб при нарушении конфиденциальности, целостности и доступности данных. Рассмотрим каждый причисленный аспект.

Если система уведомления содержит конфиденциальную информацию, такую как личные данные сотрудников, то нарушение конфиденциальности может привести к утечке персональных данных, что может иметь юридические и репутационные последствия для компании. Степень ущерба может быть средней или высокой в зависимости от чувствительности данных.

Нарушение целостности информации в системе уведомления может привести к искажению или потере важных сообщений, что может вызвать дезинформацию среди сотрудников и привести к сбоям в рабочих процессах. Степень ущерба может быть средней, если искажение информации может вызвать умеренные негативные последствия для компании.

Если система уведомления недоступна, сотрудники могут не получить важные сообщения вовремя, что может повлиять на оперативность выполнения задач и принятие решений. Степень ущерба может быть средней, если недоступность системы приводит к умеренным негативным последствиям для компании.

На основании этих оценок можно определить уровень значимости системы уведомления сотрудников компании:

- Если хотя бы для одного из свойств безопасности информации (конфиденциальности, целостности, доступности) определена высокая степень ущерба, то система имеет высокий уровень значимости (УЗ 1).
- Если хотя бы для одного из свойств безопасности информации определена средняя степень ущерба и нет ни одного свойства, для которого определена высокая степень ущерба, то система имеет средний уровень значимости (УЗ 2).
- Если для всех свойств безопасности информации определены низкие степени ущерба, то система имеет низкий уровень значимости (УЗ 3).

По всем параметрам система оповещения сотрудников компании имеет среднюю степень ущерба. В этом случае система будет иметь средний уровень значимости.

Система среднего уровня значимости и объектового масштаба должна соответствовать второму классу защищенности.

5.2 Обеспечение требований о защите

Доступ к системе имеют только аутентифицированные пользователи. Регистрация новых пользователей в системе доступна для работников отдела кадров имеющих, соответствующие права доступа.

Для защиты от несанкционированного доступа к системе должен применяться перечень технических и организационных мер.

Также повышение уровня защитности может обеспечить возможность установки системы локально, без доступа к внешней сети.

5.3 Выводы по разделу

В данном разделе был определен класс безопасности системы согласно приказу ФСТЭК от 11.02.2013 «Об утверждении требований о защите информации, не составляющей государственную тайну, содержащейся в государственных информационных системах».

Также был приведен перечень мер, используемых в системе для достижения нужного класса защищенности.

ЗАКЛЮЧЕНИЕ

В ходе написания ВКР была проанализирована и описана информационная система и её конкуренты. На основе выявленных в конкурирующих системах недостатках были выдвинуты требования к разрабатываемой системе.

В ходе проектирования системы была описана её структура, функционал и процессы, происходящие как при взаимодействии между пользователем и системой, так и между элементами самой системы. Исходя из этого была построена ER модель базы данных и прототип графического интерфейса клиентской части системы.

Был описан процесс разработки базы данных, серверной части и клиентской части приложения. Далее было произведено функциональное тестирование разработанной системы, в котором было подтверждено соответствие системы функциональным требованиям из технического задания.

В конечном итоге было приведено технико-экономическое обоснование проекта и определение класса защищенности системы.

В дальнейшем планируется работа над интерфейсом веб приложения, расширением возможностей рассылки оповещений и предоставление возможности прикрепления документов к уведомлениям.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Вигерс К., Битти Дж. Разработка требований к программному обеспечению. 3-е изд., дополненное / Пер. с англ. М.: Издательство «Русская редакция»; СПб.: БХВ Петербург, 2014.
2. Орлов С.А., Цилькер Б.Я. Технологии разработки программного обеспечения: Учебник для вузов. 4-е изд. Стандарт третьего поколения. СПб.: Питер, 2021.
3. Axios Docs. [Электронный ресурс] // Getting Started URL: <https://axios-http.com/docs/intro> (Дата обращения: 24.05.2024).
4. Express – web application framework. [Электронный ресурс] URL: <https://expressjs.com> (Дата обращения: 24.05.2024).
5. Node.js – Run JavaScript Everywhere. [Электронный ресурс] // About. URL: <https://nodejs.org/en/about> (Дата обращения: 24.05.2024).
6. Nodemailer. // About. [Электронный ресурс] URL: <https://nodemailer.com/about/> (Дата обращения: 24.05.2024).
7. npm. [Электронный ресурс] // jsonwebtoken. URL: <https://www.npmjs.com/package/jsonwebtoken> (Дата обращения: 24.05.2024).
8. npm. [Электронный ресурс] // cors. URL: <https://www.npmjs.com/package/cors> (Дата обращения: 24.05.2024).
9. npm. [Электронный ресурс] // passfather. URL: <https://www.npmjs.com/package/passfather> (Дата обращения: 24.05.2024).
10. npm. [Электронный ресурс] // bcrypt. URL: <https://www.npmjs.com/package/bcrypt> (Дата обращения: 24.05.2024).
11. npm. [Электронный ресурс] // cookie-parser. URL: <https://www.npmjs.com/package/cookie-parser> (Дата обращения: 24.05.2024).
12. PostgreSQL. [Электронный ресурс] // About. URL: <https://www.postgresql.org/about/> (Дата обращения: 24.05.2024).
13. pg-promise API. [Электронный ресурс] URL: <https://vitaly-t.github.io/pg-promise/index.html> (Дата обращения: 24.05.2024).

14. Vuex. [Электронный ресурс] // What is Vue. URL: <https://vuex.vuejs.org> (Дата обращения: 24.05.2024).

15. Vue Router. [Электронный ресурс] // Introduction. URL: <https://router.vuejs.org/introduction.html> (Дата обращения: 24.05.2024).

16. Vue.js – The Progressive JavaScript Framework. [Электронный ресурс] // Introduction. URL: <https://vuejs.org/guide/introduction.html> (Дата обращения: 24.05.2024).

ПРИЛОЖЕНИЕ А
ТЕХНИЧЕСКОЕ ЗАДАНИЕ

СОГЛАСОВАНО

Ст. преп. кафедры ИТАС

_____ Д.Б. Кузнецов

« ____ » _____ 2024

Разработку системы автоматизации уведомления сотрудников компании
для ООО «Веб депо»

Техническое задание

На 26 листах
Действует с 01.01.24

Разработал студент гр. РИС-20-26
_____ Мехоношин В.А.
« ____ » _____ 2024

СОДЕРЖАНИЕ

<u>1 ОБЩИЕ ПОЛОЖЕНИЯ</u>	82
<u>1.1 Полное наименование системы и ее условное обозначение</u>	82
<u>1.2 Наименования организации-заказчика и организаций-участников работ</u>	82
<u>1.3 Перечень документов, на основании которых создается система</u>	82
<u>1.4 Плановые сроки начала и окончания работы по созданию системы</u>	82
<u>1.5 Порядок оформления и предъявления заказчику результатов работ по созданию системы</u>	82
<u>1.6 Определения, обозначения и сокращения</u>	82
<u>2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ</u>	84
<u>2.1 Назначение системы</u>	84
<u>2.2 Цели создания системы</u>	84
<u>3 ХАРАКТЕРИСТИКА ОБЪЕКТА АВТОМАТИЗАЦИИ</u>	85
<u>3.1 Краткие сведения об объекте автоматизации</u>	85
<u>3.2 Сведения об условиях эксплуатации объекта автоматизации</u>	85
<u>4 Требования к системе</u>	86
<u>4.1 Требования к системе в целом</u>	86
<u>4.1.1 Требования к структуре и функционированию системы</u>	86
<u>4.1.2 Требования к способам и средствам связи для информационного обмена между компонентами системы</u>	86
<u>4.1.3 Требования к характеристикам взаимосвязей создаваемой АС со смежными АС, требования к ее совместимости</u>	87
<u>4.1.4 Требования по диагностированию АС</u>	87
<u>4.1.5 Перспективы развития, модернизации АС</u>	87
<u>4.2 Требования к функциям (задачам), выполняемым системой</u>	87
<u>4.2.1 Оповещение сотрудников</u>	87
<u>4.2.2 Возможность привязки реакций</u>	88
<u>4.2.3 Возможность отслеживания осведомленности сотрудников</u>	88
<u>4.2.4 Возможность просмотра уведомлений и написания к ним комментариев</u>	88
<u>4.2.5 Аутентификация и регистрация</u>	88
<u>4.2.6 Гибкость</u>	89
<u>4.3 Требования к видам обеспечения</u>	89
<u>4.3.1 Требования информационному обеспечению системы</u>	89
<u>4.3.2 Требования к лингвистическому обеспечению системы</u>	89
<u>4.3.3 Требования к программному обеспечению системы</u>	89

4.3.4 Требования к техническому обеспечению	89
4.3.5 Требования к организационному обеспечению	90
4.4 Общие технические требования к АС	90
4.4.1 Требования к численности и квалификации персонала и пользователей АС	90
4.4.2 Показатели назначения	90
4.4.3 Требования к надежности	92
4.4.4 Требования к эргономике и технической эстетике	93
4.4.5 Требования к транспортабельности для подвижных АС	93
4.4.6 Требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы	93
4.4.7 Требования к защите информации от несанкционированного доступа	93
4.4.8 Требования по сохранности информации при авариях	94
4.4.9 Требования к защите от влияния внешних воздействий	94
4.4.10 Требования к патентной частоте	94
4.4.11 Требования по стандартизации и унификации	94
5 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ (РАЗВИТИЮ) СИСТЕМЫ	95
6 Порядок разработки автоматизированной системы	96
6.1 Порядок организации разработки АС	96
6.2 Перечень документов и исходных данных для разработки АС ...	96
6.3 Перечень документов, предъявляемых по окончании соответствующих этапов работ	96
6.4 Порядок разработки, согласования и утверждения плана совместных работ по разработке АС.	96
6.5 Порядок разработки, согласования и утверждения программы работ по стандартизации.	97
6.6 Требования к гарантийным обязательствам разработчика	97
6.7 Порядок проведения технико-экономической оценки разработки АС	97
6.8 Порядок разработки, согласования и утверждения программы метрологического обеспечения, программы обеспечения надежности, программы эргономического обеспечения	97
7 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ	98
7.1 Виды, состав, объем и методы испытаний системы	98
7.2 Общие требования к приемке работ по стадиям	98
7.3 Статус приемочной комиссии	98

<u>8 ТРЕБОВАНИЯ К СОСТАВУ И СОДЕРЖАНИЮ РАБОТ ПО ПОДГОТОВКЕ ОБЪЕКТА АВТОМАТИЗАЦИИ К ВВОДУ СИСТЕМЫ В ДЕЙСТВИЕ</u>	99
<u>8.1 Подготовка серверной инфраструктуры</u>	99
<u>8.2 Тестирование и обучение персонала</u>	99
<u>9 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ</u>	100
<u>9.1 Техническая документация</u>	100
<u>10 ИСТОЧНИКИ РАЗРАБОТКИ</u>	101

1 ОБЩИЕ ПОЛОЖЕНИЯ

1.1 Полное наименование системы и ее условное обозначение

Полное наименование системы: Система оповещения сотрудников компании

Условное обозначение: АС «Система оповещения»

1.2 Наименования организации-заказчика и организаций-участников работ

Заказчик – ООО «Веб Депо»

Разработчик – студент группы РИС-20-26 Мехоношин Владислав Антонович

1.3 Перечень документов, на основании которых создается система

Система разрабатывается на основании следующих документов:

- Задание на проектирование от компании Веб Депо
- ГОСТ 34.602-2020 - Техническое задание на создание автоматизированной системы
- Федеральный закон "О персональных данных" от 27.07.2006 N 152-ФЗ

1.4 Плановые сроки начала и окончания работы по созданию системы

Начало – 02.10.23

Окончание - 27.05.24

1.5 Порядок оформления и предъявления заказчику результатов работ по созданию системы

Результаты работы предоставляются заказчику:

- Оригинальное программное обеспечение
- Отчет

1.6 Определения, обозначения и сокращения

АС – автоматизированная система.

БД – база данных.

2 НАЗНАЧЕНИЕ И ЦЕЛИ СОЗДАНИЯ СИСТЕМЫ

2.1 Назначение системы

Система предназначена для обеспечения возможности оперативного информирования сотрудников и их взаимодействия.

В её функционал должно выходить:

- Отправление уведомлений
- Просмотр списка полученных уведомлений
- Появление всплывающего окна уведомляющего о пришествии нового уведомления
- Возможность реагирования на полученные уведомления
- Отслеживание осведомленности и времени реакции сотрудников
- Возможность взаимодействия сотрудников по средствам текстовых сообщений

2.1 Цели создания системы

Система создается с целью повышения эффективности внутрифирменной коммуникации и оперативного информирования персонала о важных событиях и изменениях.

3 ХАРАКТЕРИСТИКА ОБЪЕКТА АВТОМАТИЗАЦИИ

3.1 Краткие сведения об объекте автоматизации

В ходе проведения работ по разработке Системы автоматизируются процессы по уведомлению сотрудников.

В системе должны быть некоторые заготовленные типы уведомлений (общее, задача, встреча, происшествия), которые будут сортироваться по вкладкам с аналогичными им названиями.

Так же должна быть возможность привязки возможных реакций к создаваемым уведомлениям. Реакции должны приносить гибкость в процесс уведомления сотрудников.

3.2 Сведения об условиях эксплуатации объекта автоматизации.

Описание автоматизируемых объектов: объект – это система оповещения сотрудников компании, с данными о сотрудниках и информацией о работе системы.

Описание автоматизируемых процессов: массовая рассылка уведомлений о различных происшествиях и задачах с возможностями реакции на уведомления и контроле осведомленности. Например: один из пользователей отправляет другому предложение о встрече в определенную дату, у получившего его пользователя на экране возникает окно с описанием ситуации и двумя кнопками «Принять» и «Отложить». По нажатию на одну из них отправившему это уведомление пользователю отправиться ответное уведомление с результатом. И только после реакции пользователя на уведомление будет считаться, что он о нем осведомлен.

4 ТРЕБОВАНИЯ К СИСТЕМЕ

4.1 Требования к системе в целом

4.1.1 Требования к структуре и функционированию системы

Система должна быть централизованной, т.е. все данные должны располагаться в центральном хранилище – базе данных, расположенной локально.

Должен быть реализован сервер, производящий все необходимые манипуляции с данными и производящий обмен ими с пользователями системы и базой данных.

Пользователи взаимодействуют с системой по средствам веб, десктопного или мобильного приложения.

АС оповещения сотрудников компании включает в себя следующие подсистемы:

- Уведомлений – производящая рассылку уведомлений
- Конструктор уведомлений – для созданий новых типов сценариев
- Авторизации – авторизация пользователей и разграничений прав доступа.
- Регистрации – предоставление интерфейса регистрации новых сотрудников для сотрудника отдела кадров
- Контроля осведомленности – интерфейс, предоставляющий пользователю узнать информацию об отреагировавших пользователях.

4.1.2 Требования к способам и средствам связи для информационного обмена между компонентами системы

Для информационного обмена между компонентами системы должна быть организована локальная сеть. Также должна быть возможность развернуть систему в сети интернет.

4.1.3 Требования к характеристикам взаимосвязей создаваемой АС со смежными АС, требования к ее совместимости.

Система должна быть построена на базе модульной структуры позволяющей вносить изменения и интегрировать уже существующие системы.

4.1.4 Требования по диагностированию АС.

Диагностика и профилактика технических средств, должна проводиться раз в месяц. Проверка целостности данных и проверка программного и аппаратного обеспечения проводится по мере необходимости.

4.1.5 Перспективы развития, модернизации АС.

Автоматизированная система должна иметь возможность дальнейшего модернизации как программного, так и аппаратного обеспечения.

4.2 Требования к функциям (задачам), выполняемым системой

4.2.1 Оповещение сотрудников

Отправка оповещений может производиться как одному сотруднику, так и целому отделу или сотрудникам определенной должности.

Сотрудники компании делятся на группы по их должностям, отделам.

Данные об отделах и должностях должны быть легко редактируемыми.

Оповещения могут делиться на типы, которые сортируются по следующим вкладкам в зависимости от их типа:

1. Все
2. Встреча
3. Задача
4. Происшествия
5. Общее
6. И т.п.

В каждом из полученных уведомлений должен быть указан его тип.

4.2.2 Возможность привязки реакций

При создании уведомления должна быть возможность добавления кнопок с указанием их имени и данных уведомления, которое будет отправлено по нажатию на кнопку.

4.2.3 Возможность отслеживания осведомленности сотрудников

Должна быть возможность отслеживания количества отреагировавших пользователей из общего числа получателей конкретного уведомления. Также возможность просмотра информации о состоянии реакции конкретного получателя и скорости реакции в случае отреагировавшего.

4.2.4 Возможность просмотра уведомлений и написания к ним комментариев

У пользователя должна быть возможность просмотра полученных и отправленных уведомлений и написания под ними комментариев.

4.2.5 Аутентификация и регистрация

В системе должен быть интерфейс входа и интерфейс регистрации пользователей. Интерфейс регистрации должен быть доступен только сотрудникам отдела кадров.

В системе должны храниться следующие данные пользователей:

- ФИО
- Отдел
- Должность

При регистрации должен производиться ввод этих данных, на что система должна выдавать случайно сгенерированные логин и пароль, которые будут выдаваться сотрудникам.

Регистрация и изменение данных пользователей должно происходить сотрудниками отдела кадров с заранее выданными им для этого ролей.

4.2.6 Гибкость

Должна быть возможность занесения в систему списка должностей, отделов, видов уведомлений.

4.3 Требования к видам обеспечения

4.3.1 Требования информационному обеспечению системы

Для реализации подсистемы хранения данных должна использоваться СУБД PostgreSQL.

Состав, структура и способы организации данных в системе должны быть определены на этапе технического проектирования.

Полное резервное копирование базы данных должно выполняться по расписанию раз в неделю.

4.3.2 Требования к лингвистическому обеспечению системы

Интерфейс системы должен быть на Русском языке.

4.3.3 Требования к программному обеспечению системы

Необходимо использовать свободно распространяемое программное обеспечение. Сервер системы должен корректно работать на операционной системе Linux, под дистрибутивом Debian. Также на сервер должен быть установлена СУБД PostgreSQL и среда выполнения node.js.

4.3.4 Требования к техническому обеспечению

Система должна поддерживать работу на современных серверах и виртуальных машинах.

Минимальные требования к серверу:

Процессор	Intel Xeon
ОЗУ	16ГБ
Дисковый массив	1024ГБ
Сетевые адаптеры	1000мбит

4.3.5 Требования к организационному обеспечению

Основными пользователями системы являются сотрудники компании заказчика ООО «Веб Депо». Для защиты от ошибочных действий пользователей системы должна быть предусмотрена система для проверки легитимности пользователя на возможность выполнять различные манипуляции с данными. Различные пользователи должны иметь разные уровни доступа к информации и соответственно разные возможности манипуляции с данными.

4.4 Общие технические требования к АС

4.4.1 Требования к численности и квалификации персонала и пользователей АС

Для эксплуатации автоматизированной системы (АС) определены следующие роли: системный администратор, администратор баз данных, разработчик и пользователь. Рекомендуемая численность для эксплуатации АС следующая: один разработчик, один системный администратор, один администратор баз данных, а количество пользователей определяется структурой предприятия.

Требования к квалификации персонала и пользователей АС, а также порядок их подготовки и контроля знаний и навыков:

Основные обязанности системного администратора включают:

Модернизацию, настройку и мониторинг работоспособности серверов и рабочих станций;

Установку, модернизацию, настройку и мониторинг системного и базового программного обеспечения.

Системный администратор должен обладать высокой квалификацией и практическим опытом в установке, настройке и администрировании программных и технических средств, используемых в системе.

Основные обязанности администратора баз данных включают:

- Установку, модернизацию и настройку параметров программного обеспечения СУБД;
- Оптимизацию прикладных баз данных по времени отклика и скорости доступа к данным;
- Разработку, управление и реализацию эффективной политики доступа к информации, хранящейся в прикладных базах данных.
- Администратор баз данных должен обладать высоким уровнем квалификации и практическим опытом в установке, настройке и администрировании используемых в АС СУБД.

Основные обязанности разработчика включают:

- Исправление багов и недочетов информационной системы;
- Модернизацию информационной системы.

Разработчик должен иметь полное понимание работы системы, а также знание различных фреймворков и технологий, использованных при создании системы.

Пользователи системы должны быть уверенными пользователями персонального компьютера.

Требуемый режим работы персонала и пользователей АС: Режим работы персонала с ролью администратора определяется режимом работы организации, эксплуатирующей систему, за исключением работ по устранению возможных ошибок ПО, выявленных в период опытной эксплуатации, которые проводятся по регламенту в нерабочее время.

4.4.2 Показатели назначения

Система должна функционировать в нормальном режиме при следующих обстоятельствах:

Вероятное условие	Требование
Количество одновременно работающих в системе пользователей	>100 пользователей
Количество одновременно выполняемых запросов к серверу	> 1000 запр./сек.
Время отклика при разном количестве единовременных запросов и работающих пользователей, при разном количестве обрабатываемых данных (особенно при поиске и агрегации в отчетах)	< 1000мс
Объем хранимых данных	>1000гб

4.4.3 Требования к надежности

Система должна обеспечивать высочайший уровень надежности и доступности. Время недоступности системы не должно превышать 0.1% от времени работы.

Под аварийной ситуацией понимается аварийное завершение процесса, выполняемого той или иной подсистемой, а также «зависание» этого процесса.

Надежность аппаратных и программных средств должна обеспечиваться за счет следующих организационных мероприятий:

- предварительного обучения пользователей и обслуживающего персонала
- своевременного выполнения процессов администрирования
- соблюдения правил эксплуатации и технического обслуживания программно-аппаратных средств

- своевременное выполнение процедур резервного копирования данных

4.4.4 Требования к эргономике и технической эстетике

Интерфейс должен соответствовать разработанному дизайн-проекту.

4.4.5 Требования к транспортабельности для подвижных АС

Система должна иметь конфигурационные файлы для первичной настройки.

4.4.6 Требования к эксплуатации, техническому обслуживанию, ремонту и хранению компонентов системы

Система должна быть легко обновляемой и поддерживаемой. Техническое обслуживание и ремонт компонентов системы должны выполняться квалифицированными специалистами.

4.4.7 Требования к защите информации от несанкционированного доступа

Информационная безопасность ИС должна соответствовать следующим критериям:

- Комплекс программно-технических средств и организационных мер должен обеспечивать защиту Системы.
- На всех этапах обработки информации должна быть обеспечена защита Системы.
- Средства защиты не должны значительно ухудшать основные функциональные характеристики Системы, такие как надежность, быстродействие и возможность изменения конфигурации.
- Разграничение прав доступа пользователей и администраторов должно основываться на принципе "что не разрешено, то запрещено".

4.4.8 Требования по сохранности информации при авариях

Сервера системы должны быть подключены к источникам бесперебойного питания на случай различных аварий.

Возможно резервное копирование данных в облако на случай физических повреждений серверной инфраструктуры.

4.4.9 Требования к защите от влияния внешних воздействий

Система должна быть устойчивой к воздействию внешних факторов, таких как сбой электропитания. Сервера используемые системой должны быть установлены в условиях, рекомендуемых производителем сервера или его комплектующих.

4.4.10 Требования к патентной частоте

Установка системы в целом, как и установка отдельных частей системы не должна предъявлять дополнительных требований к покупке лицензий на программное обеспечение сторонних производителей, кроме программного обеспечения, указанного в разделе 4.3.3.

4.4.11 Требования по стандартизации и унификации

Процесс разработки ИС должен соответствовать требованиям к созданию АС, регламентированных стандартами:

- ГОСТ 34.601-90 "Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания";
- ГОСТ 34.603-92 "Информационная технология. Виды испытаний автоматизированных систем".

5 СОСТАВ И СОДЕРЖАНИЕ РАБОТ ПО СОЗДАНИЮ (РАЗВИТИЮ) СИСТЕМЫ

Таблица 1 – Стадии создания автоматизированной системы.

Наименование стадии	Наименование этапа	Сроки выполнения
Формирование требований к АС	Обследование объекта и обоснование необходимости создания АС	Не позднее 1 декабря 2023 года
	Формирование требований к АС	
Эскизный проект	Разработка предварительных проектных решений по системе и ее частям	Не позднее 1 марта 2024 года
Технический проект	Разработка проектных решений по системе и ее частям	Не позднее 15 мая 2024 года
Тестирование системы	Проведение функционального тестирования	Не позднее 31 мая 2024 года

6 Порядок разработки автоматизированной системы

6.1 ПОРЯДОК ОРГАНИЗАЦИИ РАЗРАБОТКИ АС

Порядок организации разработки АС должен соответствовать порядку стадий создания автоматизированной системе, указанных ранее в таблице 1. Переход к следующим стадиям разработки, только при окончании предыдущих стадий.

6.2 Перечень документов и исходных данных для разработки АС

Основным документом для разработки АС является «Задание на проектирование от компании ООО «Веб Депо»

Также при разработке АС используются следующие документы:

- ГОСТ 34.602-2020 - Техническое задание на создание автоматизированной системы
- Федеральный закон "О персональных данных" от 27.07.2006 N 152-ФЗ

6.3 Перечень документов, предъявляемых по окончании соответствующих этапов работ

По окончанию соответствующих этапов работ предъявляются следующие документы:

- Эскизный проект
- Технический проект

6.4 Порядок разработки, согласования и утверждения плана совместных работ по разработке АС.

План совместных работ необходимо в обязательном порядке согласовывать с заказчиком. При участии нескольких организаций разработчиков, разработка должна вестись в соответствии с текущим техническим заданием.

6.5 Порядок разработки, согласования и утверждения программы работ по стандартизации.

Разработчик разрабатывает программу работ по стандартизации. Заказчик в свою очередь должен согласовать и утвердить разработанную программу.

6.6 Требования к гарантийным обязательствам разработчика

Разработчик гарантирует выполнение гарантийных обязательств в течении 5 лет с момента приема системы заказчиком.

6.7 Порядок проведения технико-экономической оценки разработки АС

Технико-экономическая оценка автоматизированной системы производится совместно с заказчиком и разработчиком после выполнения эскизного проекта, для выявления экономической эффективности применяемых технологий для разработки автоматизированной системы.

6.8 Порядок разработки, согласования и утверждения программы метрологического обеспечения, программы обеспечения надежности, программы эргономического обеспечения

Разработчик обязуется разработать программы обеспечения надежности и эргономического обеспечения, после чего программы согласуются с заказчиком и утверждаются как заказчиком, так и разработчиком.

7 ПОРЯДОК КОНТРОЛЯ И ПРИЕМКИ СИСТЕМЫ

7.1 Виды, состав, объем и методы испытаний системы

Для контроля и приемки системы будет проведено функциональное тестирование: проверка соответствия функционала требованиям.

7.2 Общие требования к приемке работ по стадиям

На каждой стадии разработки системы будет проводиться приемка работ заказчиком. Приемка будет проводиться согласно утвержденным техническим заданиям и критериям успешного выполнения работ.

7.3 Статус приемочной комиссии

Приемочная комиссия будет составлена из представителей заказчика и разработчиков. Статус комиссии будет утвержден заказчиком.

8 ТРЕБОВАНИЯ К СОСТАВУ И СОДЕРЖАНИЮ РАБОТ ПО ПОДГОТОВКЕ ОБЪЕКТА АВТОМАТИЗАЦИИ К ВВОДУ СИСТЕМЫ В ДЕЙСТВИЕ

8.2 Подготовка серверной инфраструктуры

Перед запуском системы уведомления сотрудников, заказчик должен предоставить серверное окружение, соответствующее следующим критериям:

- Виртуальный сервер с операционной системой Linux, обладающий достаточными ресурсами для обеспечения производительности системы;
- Установленное программное обеспечение, включая операционную систему, необходимые библиотеки и фреймворки для работы бота;
- Безопасные и доступные каналы связи с сервером для обеспечения регулярных обновлений и мониторинга состояния системы.

8.2 Тестирование и обучение персонала

Перед вводом системы в действие провести следующие мероприятия:

1. Провести тестирование системы с использованием тестовых данных.
2. Организовать обучение персонала, который будет обслуживать и поддерживать систему.
3. Подготовить руководства пользователя и администратора для более эффективного использования системы.

9 ТРЕБОВАНИЯ К ДОКУМЕНТИРОВАНИЮ

9.1 Техническая документация

По завершении разработки системы требуется предоставить отчет, описывающий все этапы анализа проектирования и разработки системы.

10 ИСТОЧНИКИ РАЗРАБОТКИ

При разработке данного технического задания использовались следующие источники информации:

1. Задание на проектирование от компании Веб Депо
2. ГОСТ 34.602-2020 - Техническое задание на создание автоматизированной системы
3. Федеральный закон "О персональных данных" от 27.07.2006 N 152-ФЗ

ПРИЛОЖЕНИЕ Б

SQL скрипт для создания базы данных

```
BEGIN;

CREATE TABLE IF NOT EXISTS system.employees
(
    id_employee bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    avatar text COLLATE pg_catalog."default",
    id_position bigint,
    id_department bigint,
    first_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
    second_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
    middle_name character varying(50) COLLATE pg_catalog."default",
    valid boolean DEFAULT true,
    CONSTRAINT employees_pkey PRIMARY KEY (id_employee)
);

CREATE TABLE IF NOT EXISTS system.departments
(
    id_department bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    department_name character varying(50) COLLATE pg_catalog."default" NOT
NULL,
    valid boolean DEFAULT true,
    CONSTRAINT departments_pkey PRIMARY KEY (id_department)
);

CREATE TABLE IF NOT EXISTS system.positions
(
    id_position integer NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 2147483647 CACHE 1 ),
    position_name character varying(50) COLLATE pg_catalog."default" NOT
NULL,
    valid boolean DEFAULT true,
    CONSTRAINT positions_pkey PRIMARY KEY (id_position)
);

CREATE TABLE IF NOT EXISTS system.credentials
(
    id_cred bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START
1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    password text COLLATE pg_catalog."default" NOT NULL,
    id_employee bigint NOT NULL,
    valid boolean NOT NULL DEFAULT true,
    roles json NOT NULL,
    user_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
    email character varying(50) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT credentials_pkey PRIMARY KEY (id_cred),
    CONSTRAINT unique_name UNIQUE (user_name)
    INCLUDE(user_name)
);

CREATE TABLE IF NOT EXISTS system.notifications
(
    id_notification bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT
1 START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    notification_text text COLLATE pg_catalog."default",
```

```

        id_author bigint,
        delayed boolean,
        send_date timestamp without time zone,
        id_notification_type bigint,
        drafted boolean NOT NULL,
        notification_header text COLLATE pg_catalog."default",
        CONSTRAINT notifications_pkey PRIMARY KEY (id_notification)
    );

CREATE TABLE IF NOT EXISTS system.notification_types
(
    id_type bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START
1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    type_name character varying(50) COLLATE pg_catalog."default" NOT NULL,
    valid boolean DEFAULT true,
    CONSTRAINT notification_types_pkey PRIMARY KEY (id_type)
);

CREATE TABLE IF NOT EXISTS system.notification_recipients
(
    id_recipient bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    id_notification bigint NOT NULL,
    id_employee bigint NOT NULL,
    CONSTRAINT notification_history_pkey PRIMARY KEY (id_recipient)
);

CREATE TABLE IF NOT EXISTS system.buttons
(
    id_button bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    button_text character varying(50) COLLATE pg_catalog."default" NOT NULL,
    id_notification bigint,
    id_send_notification bigint,
    CONSTRAINT buttons_pkey PRIMARY KEY (id_button)
);

CREATE TABLE IF NOT EXISTS system.sended_notifications
(
    id_sended bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    id_notification bigint NOT NULL,
    "time" timestamp with time zone NOT NULL DEFAULT now(),
    id_sender bigint NOT NULL,
    id_recipient bigint NOT NULL,
    is_reacted boolean NOT NULL DEFAULT false,
    react_time timestamp with time zone,
    CONSTRAINT sended_notifications_pkey PRIMARY KEY (id_sended)
);

CREATE TABLE IF NOT EXISTS system.notification_comments
(
    id_comment bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1
START 1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
    id_notification bigint,
    text text COLLATE pg_catalog."default",
    id_author bigint,
    "time" timestamp with time zone NOT NULL DEFAULT now(),
    CONSTRAINT notification_comments_pkey PRIMARY KEY (id_comment)
);

CREATE TABLE IF NOT EXISTS system.refresh_tokens
(

```

```

        id_token bigint NOT NULL GENERATED ALWAYS AS IDENTITY ( INCREMENT 1 START
1 MINVALUE 1 MAXVALUE 9223372036854775807 CACHE 1 ),
        refresh_token text COLLATE pg_catalog."default" NOT NULL,
        id_employee bigint NOT NULL,
        date timestamp with time zone NOT NULL DEFAULT now(),
        CONSTRAINT refresh_tokens_pkey PRIMARY KEY (id_token)
);

ALTER TABLE IF EXISTS system.employees
    ADD CONSTRAINT fk_department FOREIGN KEY (id_department)
    REFERENCES system.departments (id_department) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE
    NOT VALID;

ALTER TABLE IF EXISTS system.employees
    ADD CONSTRAINT fk_position FOREIGN KEY (id_position)
    REFERENCES system.positions (id_position) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.credentials
    ADD CONSTRAINT fk_employee FOREIGN KEY (id_employee)
    REFERENCES system.employees (id_employee) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.notifications
    ADD CONSTRAINT fk_author FOREIGN KEY (id_author)
    REFERENCES system.employees (id_employee) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.notifications
    ADD CONSTRAINT fk_type FOREIGN KEY (id_notification_type)
    REFERENCES system.notification_types (id_type) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE
    NOT VALID;

ALTER TABLE IF EXISTS system.notification_recipients
    ADD CONSTRAINT fk_notification FOREIGN KEY (id_notification)
    REFERENCES system.notifications (id_notification) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.notification_recipients
    ADD CONSTRAINT fk_recipient FOREIGN KEY (id_employee)
    REFERENCES system.employees (id_employee) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.buttons
    ADD CONSTRAINT fk_notification FOREIGN KEY (id_notification)
    REFERENCES system.notifications (id_notification) MATCH SIMPLE
    ON UPDATE CASCADE

```

```

ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.buttons
  ADD CONSTRAINT fk_send_notification FOREIGN KEY (id_send_notification)
  REFERENCES system.notifications (id_notification) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.sended_notifications
  ADD CONSTRAINT fk_notification FOREIGN KEY (id_notification)
  REFERENCES system.notifications (id_notification) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.sended_notifications
  ADD CONSTRAINT fk_recipient FOREIGN KEY (id_recipient)
  REFERENCES system.employees (id_employee) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE
  NOT VALID;

ALTER TABLE IF EXISTS system.sended_notifications
  ADD CONSTRAINT fk_sender FOREIGN KEY (id_sender)
  REFERENCES system.employees (id_employee) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE
  NOT VALID;

ALTER TABLE IF EXISTS system.notification_comments
  ADD CONSTRAINT fk_author FOREIGN KEY (id_author)
  REFERENCES system.employees (id_employee) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.notification_comments
  ADD CONSTRAINT fk_notification FOREIGN KEY (id_notification)
  REFERENCES system.notifications (id_notification) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;

ALTER TABLE IF EXISTS system.refresh_tokens
  ADD CONSTRAINT fk_id_employe FOREIGN KEY (id_employee)
  REFERENCES system.employees (id_employee) MATCH SIMPLE
  ON UPDATE CASCADE
  ON DELETE CASCADE;

END;

```

ПРИЛОЖЕНИЕ В

Код файла index.js

```
require('dotenv').config()
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser')
const authRouter = require('./router/authRouter');
const notificationRouter = require('./router/notificationRouter');
const errorMiddleware = require('./middlewares/errorMiddleware');

const PORT = process.env.PORT || 3000;
const app = express()

app.use(express.json());
app.use(cookieParser());
app.use(cors({
  credentials: true,
  origin: process.env.CLIENT_URL
}));
app.use('/auth', authRouter);
app.use('/notification', notificationRouter);
app.use(errorMiddleware);

const start = async () => {
  try {
    app.listen(PORT, () => console.log(`Server started on PORT = ${PORT}`))
  } catch (e) {
    console.log(e);
  }
}

start();
```

ПРИЛОЖЕНИЕ Г

Код хранилища employees

```
import api from '../api/index.js'
export default {
  namespaced: true,
  state: {
    employees: null
  },
  getters: {
    getEmployees(state) {
      return state.employees.filter(employee => employee.valid);
    },
    getEmployeeNameById: (state) => (id) => {
      let employee = state.employees.find(employee => employee.id_employee ==
id);
      if (employee) {
        if (employee.middle_name)
          return (employee.second_name + " " + employee.first_name + " " +
employee.middle_name);
        else {
          return (employee.second_name + " " + employee.first_name);
        }
      }else{
        return ("John Doe");
      }
    },
  },
  mutations: {
    SET_EMPLOYEES(state, payload) {
      state.employees = payload
    }
  },
  actions: {
    async loadEmployees({ commit }) {
      try {
        const response = await api.notification.getRecipients();
        console.log(response);
        commit('SET_EMPLOYEES', response.data);
      } catch (e) {
        console.log(e);
      }
    }
  }
}
```

ПРИЛОЖЕНИЕ Д

Код компонента loginPage

```
<template>
  <form @submit.prevent="logInto">
    <div class="loginForm">
      <div class="items">
        <h2>Вход</h2>
        <p v-if="error" class="errorMessage">Неверный логин или пароль</p>
        <p v-if="v$.form.login.$error" class="errorMessage">Логин должен
        быть больше 3 символов</p>
        <input type="text" v-model.trim="form.login" placeholder="Логин"
        :class="{ 'invalid': v$.form.login.$error }">
        <p v-if="v$.form.password.$error" class="errorMessage">Пароль должен
        быть больше 3 символов</p>
        <input type="password" v-model.trim="form.password"
        placeholder="Пароль" :class="{ 'invalid': v$.form.password.$error }">
        <button type="submit">Войти</button>
      </div>
    </div>
  </form>
</template>

<script>
import { useVuelidate } from '@vuelidate/core'
import { required, minLength } from '@vuelidate/validators'
export default {
  name: 'loginPage',
  setup() {
    return ({ v$: useVuelidate() })
  },
  data() {
    return {
      form: {
        login: '',
        password: ''
      },
      error: false
    }
  },
  methods: {
    async logInto() {
      this.v$.form.$touch()
      if(!this.v$.form.$error) {
        try {
          this.error = await this.$store.dispatch('user/login', this.form);
          if(!this.error) {
            this.$router.push({name: "mainPage"});
          }
        } catch(e) {
          this.error = true;
        }
      }
    }
  },
  validations: {
    form: {
      login: {required, minLength: minLength(3)},
      password: {required, minLength: minLength(3)}
    }
  },
  /*watch: {
    login(newValue) {
```

```

        if(newValue == ''){
            this.login = 'Логин';
        }
    },
    password(newValue){
        if(newValue == ''){
            this.login = 'Пароль';
        }
    }
}*/
}
</script>

<!-- Add "scoped" attribute to limit CSS to this component only -->
<style scoped>
.loginForm{
    margin-top: 15%;
    display: flex;
    flex-direction: row;
    justify-content: space-around
}
.items{
    display: flex;
    flex-direction: column;
    align-items: center;
    width: 50%;
    max-width: 400px;
    background-color: #EEEEEE;
    padding: 30px;
    border-radius: 10px;
}
.items input{
    width: 100%;
    padding: 12px 20px;
    margin: 8px 0;
    box-sizing: border-box;
    border-radius: 10px;
}
.invalid{
    border-color: red;
}
.items button{
    margin-top: 20px;
    height: 30px;
    width: 70px;
}
.items h2{
    margin: 0px;
}
.errorMessage{
    font-size: 12px;
    color: red;
    margin: 0px;
    margin-top: 5px;
}
</style>

```

ПРИЛОЖЕНИЕ Е

Код компонента mainPage

```
<template>
  <div class="container">
    <leftMenuComponent @changeType="changeType"
@newNotificationModal="openNewNotificationModal()" />
    <notificationListComponent :typeId="currnentType" v-
bind:key="currnentType" />
    <newNotificationsListComponent />
  </div>
  <newNotificationComponent v-if="newNotificationModalOpened"
@closeNotificationModal="openNewNotificationModal()" />
</template>

<script>
import leftMenuComponent from '@components/leftMenuComponent.vue';
import newNotificationComponent from
'../components/newNotificationComponent';
import notificationListComponent from
'@components/notificationListComponent.vue';
import newNotificationsListComponent from
'@components/newNotificationsListComponent.vue';
export default {
  name: 'mainPage',
  components: {
    leftMenuComponent,
    newNotificationComponent,
    notificationListComponent,
    newNotificationsListComponent,
  },
  data() {
    return {
      currnentType: 0,
      newNotificationModalOpened: false,
      timer: null
    }
  },
  created() {
    this.timer = setInterval(async () => {
      if (await
this.$store.dispatch('notifications/loadNewNotificationsNoId',
this.currnentType)) {
        await this.$store.dispatch('notificationTypes/loadUnreactedCount')
      }
      await this.$store.dispatch('commentsStore/getCommentsCount', {
notification_ids: this.$store.getters['notifications/getNotificationsIds']
});
      await this.$store.dispatch('commentsStore/getNewComments');
    }, 5000);
  },
  methods: {
    changeType(index) {
      this.currnentType = index;
    },
    openNewNotificationModal() {
      this.newNotificationModalOpened = !this.newNotificationModalOpened;
      console.log(this.newNotificationModalOpened);
    },
  },
  beforeUnmount() {
    clearInterval(this.timer)
  }
}
```

```
}  
</script>  
  
<!-- Add "scoped" attribute to limit CSS to this component only -->  
<style scoped>  
  .container {  
    display: flex;  
    height: calc(100% - 53px);  
  }  
</style>
```

ПРИЛОЖЕНИЕ Ё

Код компонента leftMenuComponent без css стилей

```
<template>
  <div v-bind:class = "(isMobile)?'left-menu mobile':'left-menu'" v-
if="this.$store.getters['leftMenu/isVisible'] || !isMobile">
    <div class="menu" v-if="this.$route.name == 'mainPage'">
      <h3>Уведомления</h3>
      <div class="bList">
        <button v-for="(type, index) in
this.$store.getters['notificationTypes/getnotificationTypes']"
@click="changeType(type.id_type)" v-bind:key="index"><span>{{
          type.type_name }}</span><span class="newNotification" v-
if="this.$store.getters['notificationTypes/getUnreactedCountByType'](type.id_
type) != 0">{{
this.$store.getters['notificationTypes/getUnreactedCountByType'](type.id_type
) }}</span></button>
        <button @click="changeType(0)"><span>Все</span></button>
      </div>
    </div>
    <div class="menu">
      <h3>Меню</h3>
      <div class="bList">
        <button @click="toMainPage()" v-if="this.$route.name !=
'mainPage'"><span>На главную страницу</span></button>
        <button @click="newNotificationModal()" v-if="this.$route.name ==
'mainPage'"><span>Новое уведомление</span></button>
        <button @click="sendedNotificationsPage()"><span>Отправленные
уведомления</span></button>
        <button @click="registerPage()" v-
if="this.$store.getters['user/getRoles'].includes('register') ||
this.$store.getters['user/getRoles'].includes('admin')"><span>Регистрация</sp
an></button>
      </div>
    </div>
  </div>
</template>

<script>
export default {
  name: 'leftMenuComponent',
  data() {
    return {
      selectedTypeId: null,
      isMobile: false
    }
  },
  async beforeCreate() {
    await this.$store.dispatch('notificationTypes/loadUnreactedCount');
  },
  created() {
    this.isMobile = ((document.documentElement.clientWidth <= 800));
    window.addEventListener('resize', this.getDimensions);
  },
  methods: {
    getDimensions() {
      this.isMobile = ((document.documentElement.clientWidth <= 800));
    },
    changeType(index) {
      this.$store.commit('leftMenu/SET_VISIBLE');
      this.$emit('changeType', index);
    },
    newNotificationModal() {
```

```

        this.$store.commit('leftMenu/SET_VISIBLE');
        this.$emit('newNotificationModal');
    },
    registerPage() {
        this.$store.commit('leftMenu/SET_VISIBLE');
        this.$router.push({name: "register"});
    },
    sendNotificationsPage() {
        this.$store.commit('leftMenu/SET_VISIBLE');
        this.$router.push({name: "sendNotificationsPage"});
    },
    toMainPage() {
        this.$store.commit('leftMenu/SET_VISIBLE');
        this.$router.push({name: "mainPage"});
    }
},
beforeUnmount() {
    window.removeEventListener('resize', this.getDimensions);
}
}
</script>

```

ПРИЛОЖЕНИЕ Ж

Код компонента notificationListComponent без css стилей

```
<template>
  <div class="listContainer">
    <div class="type">
      <h3> {{
this.$store.getters['notificationTypes/getTypeNameById'](this.typeId) }}
    </h3>
    </div>
    <div class="notificationList" ref="notifList" @scroll="listScroll">
      <notificationComponent
        v-for="(notification, index) in
this.$store.getters['notifications/getNotificationsById'](this.typeId) "
v-bind:key="index" :header="notification.notification_header"
:text="notification.notification_text"

:typeName="$store.getters['notificationTypes/getTypeNameById'](notification.i
d_notification_type)"

:senderName="$store.getters['employees/getEmployeeNameById'](notification.id_
sender)"

:sendTime="notification.time"

:buttons="$store.getters['notifications/getButtonsByNotifId'](notification.id
_notification)"
:reacted="notification.is_reacted"
:id_sended="notification.id_sended"
:id_notification="notification.id_notification" />
    </div>
  </div>
</template>

<script>
import notificationComponent from './notificationComponent.vue';
export default {
  name: 'notificationList',
  components: {
    notificationComponent
  },
  props: ['typeId'],
  async beforeCreate() {
    await this.$store.dispatch('notifications/initNotifications',
this.typeId);
    await this.$store.dispatch('commentsStore/getCommentsCount',
{notification_ids:this.$store.getters['notifications/getNotificationsIds']});
    this.$nextTick(() => {
      try {
        const list = this.$refs.notifList;
        if (list) {
          list.scrollTop = list.scrollHeight;
        }
      } catch (e) {
        console.log('no notifications');
      }
    });
  },
  methods: {
    async listScroll() {
      const list = this.$refs.notifList;
      if (list) {
        if (list.scrollTop <= 0) {
          console.log('Up');
        }
      }
    }
  }
}
```

```

        const scrollHeightBefore = list.scrollHeight; // Запоминаем текущую
        высоту скролла
        if (await
this.$store.dispatch('notifications/loadPrevNotifications', this.typeId)) {
            this.$nextTick(() => {
                const scrollHeightAfter = list.scrollHeight; // Получаем новую
                высоту скролла после подгрузки
                const scrollDifference = scrollHeightAfter -
scrollHeightBefore; // Вычисляем разницу
                list.scrollTop = scrollDifference; // Устанавливаем скролл так,
                чтобы пользователь остался на том же месте
            });
        }
    }
}
}
}
}
}
</script>

```

ПРИЛОЖЕНИЕ 3

Код компонента newNotificationListComponent

```
<template>
  <div class="newNotifListContainer">
    <newNotificationNotificationComponent
      v-for="(notification, index) in
this.$store.getters['notifications/getNewNotifications']" v-bind:key="index"
      :header="notification.notification_header"
      :text="notification.notification_text"

      :typeName="$store.getters['notificationTypes/getTypeNameById'](notification.i
d_notification_type)"

      :senderName="$store.getters['employees/getEmployeeNameById'](notification.id_
sender)"
      :sendTime="notification.time" :id_sended="notification.id_sended"
@closeModal="closeModal"

      :buttons="$store.getters['notifications/getButtonsByNotifId'](notification.id
_notification)"
      :reacted="notification.is_reacted" />
    </div>
  </template>

<script>

import newNotificationNotificationComponent from
'./newNotificationNotificationComponent.vue';
export default {
  name: 'newnNotificationListComponent',
  components: {
    newNotificationNotificationComponent,
  },
  data() {
    return {
      notifications: [],
    }
  },
  async beforeCreate() {
    await this.$store.dispatch('notifications/getLastId', this.typeId);
  },
  methods: {
    closeModal(id_sended) {

this.$store.commit('notifications/REMOVE_NOTIFICATION_NOTIFICATION',
id_sended);
    }
  }
}
</script>

<!-- Add "scoped" attribute to limit CSS to this component only -->
<style scoped>
.newNotifListContainer {
  right: 0;
  bottom: 0;
  width: 70%;
  max-width: 400px;
  position: absolute;
  margin-right: 10px;
}
</style>
```

ПРИЛОЖЕНИЕ И

Код компонента newNotificationComponent

```
<template>
  <div class="new-notificiation center">
    <h3>Новое уведомление</h3>
    <button class="closeButton" @click="closeModal"></button>
    <div class="form">
      <p class="error" v-if="buttonNameError">Названия кнопок не могут быть
пустыми</p>
      <p class="error" v-if="headerError">Заголовок не может быть пустым</p>
      <p class="error" v-if="recipientsError">Должны быть выбраны
получатели</p>
      <p class="error" v-if="selectedTypeError">У уведомления должен быть
выбран тип</p>
      <div class="block">
        <label>
          Тип уведомления:
          <select name="notifTypes" v-model="selectedType">
            <option v-bind:value="type.id_type"
              v-for="(type, index) in
$store.getters['notificationTypes/getnotificationTypes']" v-
bind:key="index">{{
              type.type_name }}</option>
            </select>
          </label>
        </div>
        <div class="block">
          <h4>Заголовок уведомления</h4>
          <textarea v-model="notificationHeader" placeholder="Введите текст
заголовка уведомления"></textarea>
        </div>
        <div class="block">
          <h4>Текст уведомления</h4>
          <textarea v-model="notificationText" placeholder="Введите текст
уведомления"></textarea>
        </div>
        <div class="block">
          <h4>Получатель</h4>
          <div class="recipients">
            <ul class="list">
              <li v-for="(department, indexD) in departmentData" v-
bind:key="indexD">
                Отдел: {{ department.department.department_name }}
                <button class="markAll" @click="markAll(`${indexD}`)">Выделить
все</button>
              </ul>
              <li v-for="(position, indexP) in department.positions" v-
bind:key="indexP">
                Должность: {{ position.position.position_name }}
                <button class="markAll"
@click="markAll(`${indexD}_${indexP}`)">Выделить все</button>
              </li>
              <li v-for="(recipient, indexR) in position.recipients" v-
bind:key="indexR">
                Сотрудник: {{ recipient.second_name }} {{
recipient.first_name }} {{ recipient.middle_name }}
                <input type="checkbox" v-bind:value="indexD + '_' +
indexP + '_' + indexR">
            </li>
          </div>
        </div>
      </div>
    </div>
  </div>
</template>
```

```

            v-bind:id="'d' + indexD + 'p' + indexP + 'r' +
indexR" v-model="recipientsCheckboxes">
        </li>
    </ul>
</li>
</ul>
</li>
</ul>
</div>
</div>
<div class="block">
    <h4>Реакции</h4>
    <div class="buttonList">
        <button @click="addButton">Добавить реакцию</button>
        <button @click="removeButton">Удалить реакцию</button>
        <div v-for="index in numOfButtonst" v-bind:key="index"
class="newButton">
            <input type="text" placeholder="Введите имя реакции"
ref="buttonName" />
        <div>
            Настройка ответа
            <button @click="showSttings(index)">Показать/Скрыть</button>
            <div v-show="visibleSettingsIds.includes(index)"
class="respSett">
                <label>
                    Тип уведомления:
                    <select name="notifTypes" ref="notifType">
                        <option v-bind:value="type.id_type"
v-for="(type, index) in
$store.getters['notificationTypes/getnotificationTypes']">
                            <div v-bind:key="index">
                                {{ type.type_name }}</div>
                        </option>
                    </select>
                </label>
                <br>
                Заголовок уведомления
                <textarea placeholder="Введите текст заголовка уведомления"
ref="headerResp"></textarea>
                Текст уведомления
                <textarea placeholder="Введите текст уведомления"
ref="textResp"></textarea>
                Получатель
                <div class="recipients">
                    <ul class="list">
                        <li v-for="(department, indexD) in departmentData" v-
bind:key="indexD">
                            Отдел: {{ department.department_name }}
                            <button class="markAll"
@click="markAll(`R${index}_${indexD}`)">Выделить все</button>
                            <ul>
                                <li v-for="(position, indexP) in
department.positions" v-bind:key="indexP">
                                    Должность: {{ position.position_name }}
                                    <button class="markAll"
@click="markAll(`R${index}_${indexD}_${indexP}`)">Выделить
                                    все</button>
                                </ul>
                                <li v-for="(recipient, indexR) in
position.recipients" v-bind:key="indexR">
                                    Сотрудник: {{ recipient.second_name }} {{
recipient.first_name }} {{ recipient.middle_name
                                    }}
                                    <input type="checkbox"

```

```

        v-bind:value="'R' + index + '_' + indexD +
        '_' + indexP + '_' + indexR"
        v-bind:id="'R' + index + 'd' + indexD + 'p' +
        indexP + 'r' + indexR"
        v-model="recipientsCheckboxes">
        </li>
    </ul>
</li>
</ul>
</li>
</ul>
</div>
</div>
</div>
</div>
</div>
</div>
</div>
<br />
<p v-if="error">Произошла ошибка при запросе</p>
<button class="sendNotification"
@click="sendNotification">Отправить</button>
</div>
</template>

<script>
export default {
  name: 'newNotificationComponent',
  data() {
    return {
      selectedType: null,
      notificationText: "",
      notificationHeader: "",
      recipientsCheckboxes: [],
      departmentsCheckboxes: [],
      positionsCheckboxes: [],
      error: false,
      numOfButtonst: 0,
      visibleSettingsIds: [],
      selectedTypeError: false,
      headerError: false,
      recipientsError: false,
      buttonNameError: false
    }
  },
  methods: {
    closeModal() {
      this.$emit('closeNotificationModal');
    },
    async sendNotification() {
      let recipients = [];
      let buttons = [];
      for (let i = 0; i < this.numOfButtonst; i++) {
        buttons.push({ buttonText: this.$refs.buttonName[i].value,
notificationHeader: this.$refs.headerResp[i].value, notificationText:
this.$refs.textResp[i].value, recipients: [], notificationType:
this.$refs.notifType[i].value });
      }
      this.recipientsCheckboxes.forEach(checkbox => {
        if (checkbox[0] !== 'R') {
          let ids = checkbox.split("_"); //department_position_employee

```

```

recipients.push(this.departmentData[ids[0]].positions[ids[1]].recipients[ids[
2]].id_employee);
    } else {
        let ids = checkbox.slice(1).split("_");
        console.log(ids);
        buttons[ids[0] -
1].recipients.push(this.departmentData[ids[1]].positions[ids[2]].recipients[i
ds[3]].id_employee);
    }
    });
    this.selectedTypeError = false;
    this.headerError = false;
    this.recipientsError = false;
    this.buttonNameError = false;
    if (recipients.length == 0)
        this.recipientsError = true;
    if (this.notificationHeader == "")
        this.headerError = true;
    if (this.selectedType == null)
        this.selectedTypeError = true;
    buttons.forEach(button => {
        if (button.recipients.length == 0)
            this.recipientsError = true;
        if (button.buttonText == "")
            this.buttonNameError = true;
        if (button.notificationType == null)
            this.selectedTypeError = true;
        if (button.notificationHeader == "")
            this.headerError = true;
    });
    if (!this.recipientsError && !this.headerError &&
!this.selectedTypeError && !this.buttonNameError) {
        try {
            await this.$api.notification.sendNotification({
                notificationHeader: this.notificationHeader,
                notificationText: this.notificationText,
                isDelayed: false, //for now
                sendDate: null,
                notificationType: this.selectedType,
                isDrafted: false,
                recipients,
                buttons
            });
            this.$emit('closeNotificationModal');
        } catch (e) {
            console.log(e);
            this.error = true;
        }
    } else {
        console.log('error');
    }
},
addButton() {
    this.numOfButtonst++;
},
removeButton() {
    if (this.numOfButtonst > 0) {
        this.numOfButtonst--;
    }
},
showSttings(id) {
    if (this.visibleSettingsIds.includes(id)) {

```

```

        this.visibleSettingsIds = this.visibleSettingsIds.filter((item) =>
            item !== id
        )
    } else {
        this.visibleSettingsIds.push(id);
    }
},
markAll(value) {
    let ids;
    if (value[0] !== 'R') {
        ids = value.split('_');
    } else {
        ids = value.slice(1).split('_');
    }
    if (value[0] !== 'R') {
        if (ids.length !== 1) {
            let len =
this.departmentData[parseInt(ids[0])].positions[parseInt(ids[1])].recipients.
length;
            for (let i = 0; i < len; i++) {
                let newCheckBox = `${ids[0]}_${ids[1]}_${i}`
                if (!this.recipientsCheckboxes.includes(newCheckBox)) {
                    this.recipientsCheckboxes.push(newCheckBox);
                }
            }
        } else {
            let lenP = this.departmentData[parseInt(ids)].positions.length;
            for (let i = 0; i < lenP; i++) {
                let lenR =
this.departmentData[parseInt(ids)].positions[parseInt(i)].recipients.length;
                for (let j = 0; j < lenR; j++) {
                    let newCheckBox = `${ids}_${i}_${j}`
                    if (!this.recipientsCheckboxes.includes(newCheckBox)) {
                        this.recipientsCheckboxes.push(newCheckBox);
                    }
                }
            }
        }
    } else {
        if (ids.length == 3) {
            let len =
this.departmentData[parseInt(ids[1])].positions[parseInt(ids[2])].recipients.
length;
            for (let i = 0; i < len; i++) {
                let newCheckBox = `R${ids[0]}_${ids[1]}_${ids[2]}_${i}`
                if (!this.recipientsCheckboxes.includes(newCheckBox)) {
                    this.recipientsCheckboxes.push(newCheckBox);
                }
            }
        } else {
            let lenP = this.departmentData[parseInt(ids[1])].positions.length;
            for (let i = 0; i < lenP; i++) {
                let lenR =
this.departmentData[parseInt(ids[1])].positions[parseInt(i)].recipients.lengt
h;
                for (let j = 0; j < lenR; j++) {
                    let newCheckBox = `R${ids[0]}_${ids[1]}_${i}_${j}`
                    if (!this.recipientsCheckboxes.includes(newCheckBox)) {
                        this.recipientsCheckboxes.push(newCheckBox);
                    }
                }
            }
        }
    }
}
}

```

```

    }
  },
  computed: {
    departmentData() { //kill it with fire!!!
      let data = [];
      let watchedPositions = []
      let departments = this.$store.getters['departments/getDepartments'];
      let positions = this.$store.getters['positions/getPositions'];
      let recipients = this.$store.getters['employees/getEmployees'];
      if (departments && positions && recipients) {
        departments.forEach(department => {
          let departmentData = {
            department: department,
            positions: []
          };
          let depRec = recipients.filter(recipient => recipient.id_department
== department.id_department);
          depRec.forEach(recipient => {
            let position = positions.find(position => position.id_position ==
recipient.id_position);
            if (!watchedPositions.includes(position)) {
              watchedPositions.push(position);
              let positionData = {
                position: position,
                recipients: depRec.filter(recipient => recipient.id_position
== position.id_position)
              };
              departmentData.positions.push(positionData);
            }
          });
          data.push(departmentData);
          watchedPositions = [];
        });
      }
      console.log(data);
      return data;
    }
  },
  watch: {
    recipientsCheckboxes(val) {
      console.log(val);
    }
  }
}
</script>

```

```

<!-- Add "scoped" attribute to limit CSS to this component only -->
<style scoped>
.new-notificiation {
  background-color: rgb(235, 235, 235);
  border-radius: 10px;
  padding: 30px;
  width: 70%;
  height: auto;
}

.center {
  position: absolute;
  left: 50%;
  top: 50%;
  transform: translate(-50%, -50%);
}

```

```

ul {
  list-style-type: none;
}

h3 {
  display: inline-block;
  margin-top: 0px;
}

h4 {
  margin: 5px;
}

.list {
  margin: 0px;
}

.closeButton {
  float: right;
  border: 0;
  background: none;
}

.closeButton:hover {
  background-color: #cacaca;
}

.closeButton:active {
  background-color: #bebebe;
}

.recipients {
  background-color: #dddddd;
  padding: 10px;
  overflow: auto;
  height: 150px;
}

.recipients .list {
  padding: 0px;
}

textarea {
  width: 98%;
  height: 70px;
  resize: none;
  overflow: auto;
}

.buttonList {
  background-color: rgb(221, 221, 221);
  border-radius: 10px;
  padding: 7px;
}

.respSett {
  background-color: rgb(194, 193, 193);
  border-radius: 10px;
  padding: 7px;
}

.newButton {

```

```
background-color: rgb(206, 206, 206);
border-radius: 10px;
padding: 7px;
margin: 3px;
}

.block {
background-color: rgb(221, 221, 221);
border-radius: 10px;
padding: 5px;
margin: 3px;
}

.form {
height: auto;
overflow: auto;
max-height: 700px;
}

.sendNotification {
width: 100%;
height: 30px;
}

.error{
font-size: 12px;
color: red;
margin: 0px;
margin-top: 5px;
}
</style>
```

ПРИЛОЖЕНИЕ Й

Код компонента `sendedNotificationsPage`

```
<template>
  <div class="sendedNotificationsPage">
    <leftMenuComponent />
    <div class="listContainer" ref="notifList" @scroll="listScroll">
      <div class="sendedNotification"
        v-for="(notification, indexN) in
this.$store.getters['sendedNotifications/getData']"
        v-bind:key="indexN">
        <p class="notificationHeader">{{
notification.notification_header }}</p>
        <pre v-if="notification.notification_text">{{
notification.notification_text }}</pre>
        <div class="info">
          <span>{{ getTimeString(notification.send_time)
}}</span>

          </div>
          <div class="comments">
            <div class="commentsHeader"><span>Комментарии
              ({{
this.$store.getters['commentsStore/getCommentsCountById'](notification.id_not
ification)

              }}</span><button
@click="showComments(notification.id_notification)">Показать</button>
            </div>
            <div class="commentsContainer"
              v-
if="this.$store.getters['commentsStore/getUpdateIds'].includes(notification.i
d_notification)">
              <div class="commentList">
                <div class="comment"
                  v-for="(comment, indexC) in
this.$store.getters['commentsStore/getCommentsByNotifId'](notification.id_not
ification)"
                  :key="indexC">
                  <p>{{ comment.text }}</p>
                  <div class="info">
                    <span>{{ getTimeString(comment.time)
}}</span>

                    <span>{{
this.$store.getters['employees/getEmployeeNameById'](comment.id_author)
                    }}</span>
                  </div>
                </div>
              </div>
              <div class="controls">
                <textarea v-model="commentText"
placeholder="Введите комментарий"></textarea><button
class="commentSend"
@click="send(notification.id_notification)">Отправить</button>
              </div>
            </div>
            <p class="notificationSendTime">Отреагировали: {{
notification.recipients_data.reduce((acc, curr) => acc
+ (curr.state ? 1 : 0), 0) }}/{{
notification.recipients_data.length }}</p>
            <button class="showDataButton"
@click="showData(indexN)">Показать/Скрыть информацию о
```

```

        получателях</button>
        <div class="recipientsData" v-
if="showContentList.includes(indexN)">
            <p v-for="(recipient, indexR) in
notification.recipients_data" v-bind:key="indexR"> {{
this.$store.getters['employees/getEmployeeNameById'](recipient.employee) }}
<span
class="reacted" v-
if="recipient.state">отреагировал</span><span class="notReacted"
v-if="!recipient.state"> не
отреагировал</span> <span v-
if="recipient.react_time">через {{ Math.round(((new
Date(recipient.react_time)) - (new
Date(notification.send_time))) / (60 * 1000)) }}мин. с
момента отправки</span></p>
        </div>
    </div>
</div>
<newNotificationsListComponent />
</div>
</template>

<script>
import newNotificationsListComponent from
'@/components/newNotificationsListComponent.vue';
import leftMenuComponent from '@components/leftMenuComponent.vue';
export default {
    name: 'sendedNotificationsPage',
    components: {
        newNotificationsListComponent,
        leftMenuComponent,
    },
    data() {
        return {
            timer: null,
            showContentList: [],
            commentText: '',
        }
    },
    methods: {
        mainPage() {
            this.$router.push({ name: "mainPage" });
        },
        showData(indexN) {
            if (!this.showContentList.includes(indexN)) {
                this.showContentList.push(indexN);
            } else {
                this.showContentList = this.showContentList.filter(elem
=> elem !== indexN);
            }
        },
        async showComments(id_notification) {
            if
(!this.$store.getters['commentsStore/getUpdateIds'].includes(id_notification)
) {
                await this.$store.dispatch('commentsStore/getComments',
{ id_notification: id_notification });
                this.$store.commit('commentsStore/ADD_UPDATE_ID',
id_notification);
            } else {
                this.$store.commit('commentsStore/DELETE_UPDATE_ID',
id_notification);
            }
        }
    }
}

```

```

    },
    async send(id_notification) {
      if (await this.$store.dispatch('commentsStore/sendComment',
{ id_notification, text: this.commentText, id_author:
this.$store.getters['user/getUser'].id_employee }))
        this.commentText = '';
    },
    async listScroll() {
      const list = this.$refs.notifList;
      if (list) {
        if (Math.abs(list.scrollHeight - list.scrollTop -
list.clientHeight) < 1) {
          console.log('Down');
          const scrollHeightBefore = list.scrollHeight; //
Запоминаем текущую высоту скролла
          if
(this.$store.dispatch('sendedNotifications/getPrevData')) {
            this.$nextTick(() => {
              const scrollHeightAfter = list.scrollHeight;
// Получаем новую высоту скролла после подгрузки
              const scrollDifference = scrollHeightAfter +
scrollHeightBefore; // Вычисляем разницу
              list.scrollTop = scrollDifference; //
Устанавливаем скролл так, чтобы пользователь остался на том же месте
            });
          }
        }
      }
    },
    getTimeString(sendTime) {
      let time = new Date(sendTime);

      return `${("0" + time.getDate()).slice(-2)}${("0" +
(time.getMonth() + 1)).slice(-2)}${time.getFullYear()}${("0" +
time.getHours()).slice(-2)}:${("0" + time.getMinutes()).slice(-2)}`;
    },
    created() {
      this.$store.dispatch('sendedNotifications/initData');
      this.timer = setInterval(async () => {
        if
this.$store.dispatch('notifications/loadNewNotificationsNoId',
this.currentType)) {
          await
this.$store.dispatch('notificationTypes/loadUnreactedCount')
        }
        await
this.$store.dispatch('sendedNotifications/getUpdateData');
        await this.$store.dispatch('commentsStore/getCommentsCount',
{ notification_ids: this.$store.getters['sendedNotifications/getUpdateIds']
});
        await this.$store.dispatch('commentsStore/getNewComments');
      }, 5000);
    },
    beforeUnmount() {
      clearInterval(this.timer)
    },
  }
}
</script>

```

ПРИЛОЖЕНИЕ К

Код компонента registerPage

```

<template>
  <div class="registerPage">
    <leftMenuComponent />
    <div class="container">
      <div class="label">
        <h2>Регистрация новых сотрудников</h2>
      </div>
      <div class="registerForm">
        <p class="error" v-if="v$.department.$error">Укажите
департамент</p>
        <label>
          Отдел:
          <select name="departmentSelection" v-
model="department">
            <option v-bind:value="department.id_department"
              v-for="(department, index) in
$store.getters['departments/getDepartments']"
              v-bind:key="index">{{
department.department_name }}</option>
          </select>
        </label>
        <p class="error" v-if="v$.position.$error">Укажите
должность</p>
        <label>
          Должность:
          <select name="positionSelection" v-model="position">
            <option v-bind:value="position.id_position"
              v-for="(position, index) in
$store.getters['positions/getPositions']" v-bind:key="index">{{
position.position_name }}</option>
          </select>
        </label>
        <p class="error" v-if="v$.firstName.$error">Укажите
имя</p>
        <label>
          Имя:
          <input type="text" placeholder="Введите имя" v-
model="firstName" />
        </label>
        <p class="error" v-if="v$.secondName.$error">Укажите
фамилию</p>
        <label>
          Фамилия:
          <input type="text" placeholder="Введите фамилию" v-
model="secondName" />
        </label>
        <label>
          Отчество:
          <input type="text" placeholder="Введите отчество" v-
model="thridName" />
        </label>
        <p class="error" v-if="v$.emailValue.$error">Укажите
правильную электронную почту</p>
        <label>
          Элктронная почта:
          <input type="text" placeholder="Введите электронную
почту" v-model="emailValue" />
        </label>

```

```

        <button class="send" @click="send">Отправить</button>
    </div>
    <div class="label">
        <h2>Изменение данных</h2>
    </div>
    <div class="buttons"><button class="update"
@click="update">Обновить</button>
        <button class="sendUpdated" @click="sendUpdated">Отправить
обновление</button></div>
        <div class="usersData" ref="usersList" @scroll="listScroll"
v-bind:key="ididid">
            <div v-for="(userData, indexU) in
this.$store.getters['usersData/getUsers']" v-bind:key="indexU"
                class="userData">
                    <label>Имя: <input type="text" placeholder="введите
имя"
                                v-bind:oninput="this.firstNameChangeHandler"
v-bind:value="userData.first_name"
                                v-bind:id="userData.id_employee + '
f'"></label>
                    <label>Фамилия: <input type="text"
placeholder="введите фамилию"
                                v-
bind:oninput="this.secondNameChangeHandler" v-
bind:value="userData.second_name"
                                v-bind:id="userData.id_employee + '
s'"></label>
                    <label>Отчество: <input type="text"
placeholder="введите отчество"
                                v-
bind:oninput="this.middleNameChangeHandler" v-
bind:value="userData.middle_name"
                                v-bind:id="userData.id_employee + '
t'"></label>
                    <label>
                        Отдел:
                        <select name="departmentSelection" v-
bind:id="userData.id_employee + ' d'" v-
bind:oninput="this.departmentChangeHandler">
                            <option v-
bind:value="department.id_department"
                                v-
bind:selected="(department.id_department == userData.id_department) ?
'selected' : null"
                                v-for="(department, index) in
$store.getters['departments/getDepartments']"
                                v-bind:key="index">{{
department.department_name }}</option>
                        </select>
                    </label>
                    <label>
                        Должность:
                        <select name="positionSelection" v-
bind:id="userData.id_employee + ' p'" v-
bind:oninput="this.positionChangeHandler">
                            <option v-bind:value="position.id_position"
v-bind:selected="(position.id_position
== userData.id_position) ? 'selected' : null"
                                v-for="(position, index) in
$store.getters['positions/getPositions']"
                                v-bind:key="index">{{
position.position_name }}</option>
                        </select>

```

```

        </label>
        <label>Поли: <input type="text" placeholder="введите
поли" v-bind:value="userData.roles" v-bind:id="userData.id_employee + ' r'"
v-bind:oninput="this.rolesChangeHandler"></label>
        <label>Зарегистрирован:
{{userData.registered}}</label>
        <label>Работает: <input type="text" v-
bind:value="userData.valid" v-bind:id="userData.id_employee + ' w'" v-
bind:oninput="this.worksChangeHandler"></label>
    </div>
</div>
</div>
<newNotificationsListComponent />

</div>
</template>

<script>
import newNotificationsListComponent from
'@/components/newNotificationsListComponent.vue';
import leftMenuComponent from '@components/leftMenuComponent.vue';
import { useVuelidate } from '@vuelidate/core'
import { required, email } from '@vuelidate/validators'
export default {
    name: 'registerPage',
    components: {
        newNotificationsListComponent,
        leftMenuComponent,
    },
    setup() {
        return ({ v$: useVuelidate() })
    },
    data() {
        return {
            firstName: null,
            secondName: null,
            thridName: null,
            position: null,
            department: null,
            emailValue: null,
            timer: null,
            ididid: 0
        }
    },
    methods: {
        firstNameChangeHandler(event) {
            this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'firstName'});
        },
        secondNameChangeHandler(event) {
            this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'secondName'});
        },
        middleNameChangeHandler(event) {
            this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'middleName'});
        }
    }
}

```

```

    },
    departmentChangeHandler(event) {

this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'department'}));
    },
    positionChangeHandler(event) {

this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'position'}));
    },
    rolesChangeHandler(event) {

this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'roles'}));
    },
    worksChangeHandler(event) {

this.$store.commit('usersData/SET_USER_UPDATED',{id_employee:
event.srcElement.id.split(' ')[0], value: event.target.value, changed:
'works'}));
    },
    async send() {
      this.v$.stouch()
      if (!this.v$.error) {
        try {
          let payload = { first_name: this.firstName,
second_name: this.secondName, middle_name: this.thridName, id_position:
this.position, id_department: this.department, email: this.emailValue };
          const result = await
this.$api.auth.register(payload);
          if (result === 'ok') {
            alert('Пользователь зарегистрирован');
          } else {
            alert('Этот email уже привязан к учетной
записи');
          }
        } catch (e) {
          console.log(e);
        }
      }
    },
    update() {
      this.$store.dispatch('usersData/getUsersData');
      this.ididid++;
    },
    async sendUpdated() {
      let ok = await
this.$store.dispatch('usersData/sendNewUsersInfo');
      if(ok){
        this.$store.dispatch('usersData/getUsersData');
        this.ididid++;
      }
    },
    async listScroll() {
      const list = this.$refs.usersList;
      if (list) {
        if (Math.abs(list.scrollHeight - list.scrollTop -
list.clientHeight) < 1) {
          console.log('Down');

```

```

        const scrollHeightBefore = list.scrollHeight; //
Запоминаем текущую высоту скролла
        if
(this.$store.dispatch('usersData/getPrevUsersData')) {
            this.$nextTick(() => {
                const scrollHeightAfter = list.scrollHeight;
// Получаем новую высоту скролла после подгрузки
                const scrollDifference = scrollHeightAfter +
scrollHeightBefore; // Вычисляем разницу
                list.scrollTop = scrollDifference; //
Устанавливаем скролл так, чтобы пользователь остался на том же месте
            });
        }
    },
    created() {
        this.$store.dispatch('usersData/getUsersData');
        this.timer = setInterval(async () => {
            if (await
this.$store.dispatch('notifications/loadNewNotificationsNoId',
this.currentType)) {
                await
this.$store.dispatch('notificationTypes/loadUnreactedCount')
            }
        }, 5000);
    },
    beforeUnmount() {
        clearInterval(this.timer)
    },
    validations: {
        firstName: { required },
        secondName: { required },
        position: { required },
        department: { required },
        emailValue: { required, email },
    }
}
</script>

<!-- Add "scoped" attribute to limit CSS to this component only -->
<style scoped>
.label {
    border-bottom: solid;
}

.registerPage {
    height: calc(100% - 53px);
    display: flex;
}

.container {
    width: 100%;
    display: flex;
    align-items: center;
    flex-direction: column;
}

.registerForm {
    display: flex;
    flex-direction: column;
    background-color: #cccccc;

```

```

padding: 10px;
margin: 5px;
border-radius: 8px;
width: 90%;
max-width: 600px;
}

label {
background-color: #bdbdbd;
border-radius: 8px;
padding: 5px;
margin: 3px;
}

h2 {
margin: 0px;
}

.label {
text-align: center;
background-color: #c7c7c7;
border-radius: 10px;
padding: 5px;
margin: 5px;
width: 90%;
}

.error {
font-size: 12px;
color: red;
margin: 0px;
margin-top: 5px;
}

.usersData {
width: 100%;
height: calc(100% - 397px);
overflow: auto;
width: 70%;
}

.userData {
margin: 8px;
display: flex;
flex-direction: column;
background-color: #cccccc;
padding: 3px;
border-radius: 10px;
}

.userData label{
padding: 5px;
}

.buttons{
display: flex;
background-color: #c7c7c7;
border-radius: 10px;
padding: 5px;
margin: 5px;
width: 90%;
}

.buttons button{
width: 100%;

```

```
        margin: 3px;  
    }  
</style>
```